МИНОБРНАУКИ РОССИИ

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет «Московский институт электронной техники»

Кафедра «Информационная безопасность»

КУРСОВАЯ РАБОТА

ПО ДИСЦИПЛИНЕ

«МЕТОДЫ И СРЕДСТВА КРИПТОГРАФИЧЕСКОЙ ЗАЩИТЫ ИНФОРМАЦИИ»

ТЕМА: «СРАВНИТЕЛЬНЫЙ АНАЛИЗ ШИФРОВ ГАММИРОВАНИЯ»

Студент ИБ-31 учебной группы _____ (Бройдо Г.Д.)

подпись *(фамилия, инициалы)*

2025 г.

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ

«Национальный исследовательский университет «МИЭТ»

Кафедра «Информационная безопасность» (ИБ)

«УТВЕРЖДАЮ»

Заведующий кафедрой ИБ

_____ А.А.Хорев

« _____ » _____ 2025 г.

ЗАДАНИЕ

НА КУРСОВУЮ РАБОТУ

по дисциплине «Криптографические методы защиты информации»

Студенту учебной группы ИБ-31 Бройдо Глебу Дмитриевичу .

фамилия, имя , отчество

Тема курсовой работы: «Сравнительный анализ шифров гаммирования с элементами криптоанализа (тема № 3)».

Целевая установка и исходные данные: На основе результатов шифрования литературного текста выбранными шифрами и исследования способов их криптоанализа провести сравнительный анализ выбранных шифров по трудоемкости и криптостойкости и дать рекомендации по их применению для защиты информации.

Основные вопросы, подлежащие разработке:

1. Выбор шифров для проведения сравнительного анализа по заданной теме.
2. Анализ алгоритмов шифрования, как способов криптографической защиты информации.
3. Определение места исследуемых шифров в классификационной схеме и дереве развития криптографии.
4. Разработка блок-схем алгоритмов шифрования и расшифрования.
5. Разработка программ в соответствие с блок-схемами.
6. Отладка программ по тестам ручного шифрования.
7. Организация общего интерфейса программ.
8. Шифрование и расшифрование выбранного текста (не менее 1000 знаков) с помощью разработанных программ шифрования.
9. Построение гистограмм исходного и шифрованных текстов, диаграмм символов, биграмм и триграмм.
10. Анализ результатов сравнения гистограмм и диаграмм.
11. Криптоанализ зашифрованных текстов.
12. Разработка алгоритмов криптоанализа.
13. Построение блок-схемы алгоритмов.
14. Сравнительный анализ криптостойкости исследуемых шифров.

15. Выработка рекомендаций по использованию для защиты информации.

К защите представить: пояснительную записку объемом 30 – 50 страниц с приложением исполняемого файла (на машинном носителе) и текстов программ с комментариями.

Основная рекомендуемая литература и материалы:

1. Бутакова Н.Г., Семененко В.А., Федоров Н.В. Криптографическая защита информации: Учебное пособие. – М.:МГИУ, 2011. – 316 с.
2. Алферов А.П., Зубов А.Ю., Кузьмин А.С., Черемушкин А.В. Основы криптографии: Учебное пособие. – М.: Гелиос АРВ, 2002. – 480 с.
3. Зубов А.Ю. Совершенные шифры.- М: Гелиос АРВ , 2003.- 162 с.
4. Словарь криптографических терминов / Под редакцией Б.А.Погорелова и В.Н.Сачкова.- М: МЦНМО, 2006.
5. Введение в криптографию / Под общей редакцией В.В.Ященко. – 3-е изд., доп. – М.: МЦНМО: «ЧеРо», 2000. – 288 с.
6. Нечаев В.И. Элементы криптографии (Основы теории защиты информации): Учеб. пособие для ун-тов и пед. Вузов / Под ред. В.А.Садовничего – М.: Высш.шк., 1999, - 109 с.
7. Виноградов И.М. Основы теории чисел. – 10-е изд., стер. – СПб.: Издательство «Лань», 2004. – 176 с.
8. Жданов О.Н., Чалкин В.А. Эллиптические кривые: Основы теории и криптографические приложения.-М.: Книжный дом «ЛИБРОКОМ», 2013.-200 с. (Основы защиты информации.)
9. Масленников М. Практическая криптография. – СПб.: БХВ-Петербург, 2003. – 464 с.
10. Коутинхо С.. Введение в теорию чисел. Алгоритм RSA. – М.: Постмаркет, 2001. – 328 с.
11. Бабаш А.В., Шанкин Г.П. История криптографии. Ч.1. – М.: Гелиос АРВ, 2002. – 240 с.
12. Грушо А.А.. Тимонина Е.Е. Теоретические основы защиты информации. -М.: Издательство Агентства «Яхтсмен». 1996. - 192 с.
13. Бутакова Н.Г.Основы защиты конфиденциальных документов: Учебное пособие. - М.:МОСУ, 2005. – 246 с.
14. Семененко В.А., Бутакова Н.Г. Основы защиты информации в компьютерных системах: Учебное пособие. – М.: МОСУ, 2004. – 238 с.
15. Закон «Об информации, информационных технологиях и защите информации» от 27 июля 2006 года N 149-ФЗ.
16. Федеральный Закон “Об электронной цифровой подписи” от 10 января 2002 г. N 1-ФЗ.
17. ГОСТ 28147-89. Системы обработки информации. Защита криптографическая. Алгоритм криптографического преобразования.- М., Госстандарт, 1990.
18. ГОСТ Р 34.10-2012. Информационная технология. Криптографическая защита информации. Процессы формирования и проверки электронной цифровой подписи.- М., Госстандарт, 2001.
19. ГОСТ Р 34.11-2012. Функция хеширования.- М., Госстандарт, 1994.
20. Соболева Т.А. История шифровального дела в России. - М.: ОЛМА-ПРЕСС-Образование, 2002.
21. КомпьютерПресс: Безопасность, 2003, №3 (март).
22. Романец Ю.В., Тимофеев П.А., Шаньгин В.Ф. Защита информации в компьютерных системах и сетях. – М.: Радио и связь, 2001. – 376 с.
23. Шнайер Б. Прикладная криптография. Протоколы, алгоритмы, исходные тексты на языке Си. – М.: ТРИУМФ, 2002 – 816 с.
24. Баричев С.Г., Гончаров В.В., Серов Р.Е. Основы современной криптографии – Москва, Горячая линия – Телеком, 2001
25. Анохин М. И., Варновский Н. П., Сидельников В. М., Ященко В. В. «Криптография в банковском деле». М. Изд-во МИФИ, 1997
26. Крысин А.В. Информационная безопасность: Практическое руководство. – М.: «СПАРРК», К.: «ВЕК+», 2003. – 320 с.
27. Жельников В. Криптография от папируса до компьютера - М., 1996
28. Теория и практика обеспечения информационной безопасности. Под редакцией Зегжды П.Д. - М., Яхтсмен, 1996.
29. Кочер П. Временной анализ реализаций Диффи-Хеллмана, RSA, DSS и других систем

30. Mark W. Eichin, Jon A. Rochils. With Microscope and Tweezers: An Analysis of the Internet virus of November 1988.
31. Шнайер Б, Мюдж П. Криптоанализ протокола PPTP от Microsoft.
32. Eli Biham, Lars R. Knudsen. Cryptanalysis of the ANSI X9.52 CBCM Mode. Proceedings of Eurocrypt'98.
33. КомпьютерПресс 3'2002
34. Винокуров А.Ю. ГОСТ не прост, а очень прост, М., Монитор.–1995.–N1.
35. Винокуров А.Ю. Еще раз про ГОСТ.-М., Монитор.–1995.–N5.
36. Смид М.Э., Бранстед Д.К. Стандарт шифрования данных: прошлое и будущее. /пер. с англ./ М., Мир, ТИИЭР.–1988.–т.76.–N5.
37. Б.В.Березин, П.В.Дорошкевич. Цифровая подпись на основе традиционной криптографии//Защита информации, вып.2.,М.: МП "Ирбис-II",1992.
38. W.Diffie,M.E.Hellman. New Directions in cryptography// IEEE Trans. Inform. Theory, IT-22, vol 6 (Nov. 1976), pp. 644-654.
39. У.Диффи. Первые десять лет криптографии с открытым ключом. /пер. с англ./ М., Мир, ТИИЭР.–1988.–т.76.–N5.
40. Водолазкий В., Стандарт шифрования ДЕС, Монитор 03-04 1992 г. С.
41. Воробьев С. Защита информации в персональных ЗВМ.-М:Мир, 1993 г.
42. Ковалевский В.Криптографические методы, Компьютер Пресс 05.93 г.
43. Мафтик С. Механизмы защиты в сетях ЭВМ. -М.: Мир, 1993. -216 с.
44. Фролов Г. Тайна тайнописи. - М., 1992.
45. Зелинский Ф. Сказочная древность Эллады. - М., 1993.
46. Жельников В. Криптография от папируса до компьютера. М., 1996.
47. Горбовский А. Загадки древнейшей истории. - М., 1971.
48. Пазин С.В. Основы защиты информации в компьютерных системах. – М.: ТВП/ОпиПМ, 2003.
49. Стрельцов А.А. Обеспечение информационной безопасности России. Теоретические и методические основы / Под ред. В.А.Садовничего и В.П.Шерстюка – М.: МЦНМО, 2002.
50. Информационная безопасность и защита информации. Учебное пособие. /Степанов Е.А. и др. - М.: Инфра-М, 2002.
51. Информационная безопасность. Учебное пособие. Ярочкин В.И. - М.: Международные отношения, 2000.
52. Малюк А.А., Пазизин С.В., Погожин Н.С. Введение в защиту информации в автоматизированных системах: Учебное пособие. – М.: Горячая линия - Телеком, 2001.
53. Завгородний В.И. Комплексная защита информации в компьютерных системах: Учебное пособие. – М.: Логос, 2001.
54. Зегжда Д.П., Ивашко А.М. Основы безопасности информационных систем. – М.: Горячая линия – Телеком, 2000.
55. Девянин П.М., Михальский Д.И., Правиков Д.И., Щербаков А.Ю. Теоретические основы компьютерной безопасности: Учеб. пособие для вузов. – М.: Радио и связь, 2000.
56. Артехин Б.В. Стеганография // Защита информации. Конфидент. - 1996. -№4.
57. Барсуков В.С. Обеспечение информационной безопасности. - М: ТЭК, 1996.
58. Галатенко В.. Трифаленков И. Информационная безопасность в Интранет: концепции и решения // Jet Info. №23/24, 1996.
59. Герасименко В.А. Защита информации в автоматизированных системах обработки данных: В 2 кн.. М.: Энергоатомиздат, 1994.
60. Гостехкомиссия РФ. Руководящий документ. Защита информации. Специальные защитные знаки. - М.: Jet Info, 1997.
61. Гостехкомиссия РФ. Руководящий документ. Средства вычислительной техники. Межсетевые экраны. Защита от несанкционированного доступа к информации. Показатели защищенности от несанкционированного доступа к информации. - М.: Jet Info, 1997.
62. Гостехкомиссия РФ. Руководящий документ. Концепция защиты средств вычислительной техники и автоматизированных систем от несанкционированного доступа к информации. - М.: Jet Info, №2,1996.
63. Груздев С.Л., Хачатурова О.Л. Электронные ключи «YASP» компании «ALADDIN». Новые технологии в маркетинге программного обеспечения //Защита информации. Конфидент. - 1996. - №6.

64. Защита программного обеспечения: Пер. с англ./ Д.Гроувер, Р.Сатер, Дж-Фипс и др.; Под ред. Д.Гроувера. - М.: Мир. 1992. - 286 с.
65. Алешин И. и др. Информационно-безопасные системы. Анализ проблемы: Учеб. пособие / Под ред. В.Н. Козлова - СПб.: Издательство С.-Петербургского гос. техн. университета. 1996. -69 с.
66. Лебедев А.Н. Криптография с «открытым ключом» и возможности его практического применения // Защита информации. Конфидент. - 1992. - №2.
67. Медведовский И.Д., Семьянов П.В., Платонов В.В. Атака через INTERNET /Под научной редакцией проф. П.Д. Зегжды - СПб.: Мир и семья, 1997. -296с.
68. Мельников В.В. Защита информации в компьютерных системах. - М.: Финансы и статистика; Электронинформ, 1997. - 368 с.
69. Михайлов С.Ф., Петров В.А., Тимофеев Ю. А. Информационная безопасность. Защита информации в автоматизированных системах. Основные концепции: Учебное пособие. - М.: МИФИ, 1995. - 112 с.
70. Протоколы и методы управления в сетях передачи данных / Пер. с англ.: Под ред. Ф.Ф.Куо. - М.: Радио и связь, 1985.
71. Стенг Д.И., Мун С. Секреты безопасности сетей. - К.: Диалектика, 1996.
72. Хоффман Л.Дж. Современные методы защиты информации / Пер. с англ. -М: Сов. радио, 1980.
73. Шеннон К. Работы по теории информации и кибернетике. - М.: ИИЛ, 1963. - 830с.
74. Шрейдер Ю.А. О семантических аспектах теории информации. Информация и кибернетика. - М.: Сов. радио, 1967.
75. Хоффман Л. Дж. Современные методы защиты информации, М.: Сов..
76. Оре О. Приглашение в теорию чисел. – Пер. с англ. Изд. 2-е, стер. – М.: Едиториал УРСС, 2003. – 128 с.
77. Digital signature standart (DSS). FIPS PUB 190, 1994, MAY 19.
    a. Massey J. Cryptography: Fundamentals and Applications. Advanced Technology Seminars, 1994.
78. Menzes A., Oorschot P., Vanstone S. Hadbook of Applied Cryptography, CRC Press, 1997.
79. Meyer C., Matyas S. Cryptography: a New Dimension in Computer Data Security. John Wiley & Sons, 1982.
80. Salomaa A. Public-key cryptography, Springer-Verlas, Berlin. Heidelbers New York, London, Paris, Tokyo, Hong Kong, Barselona, 1990.
81. Schneier B. Applied Cryptography Second Edition: protocols, algorithms and source code in C. John Wiley & Sons Inc., 1996.
82. Амамия М., Танака Ю. Архитектура ЭВМ и искусственный интеллект. М.: Мир, 1993.
83. Гилл А. Линейные последовательные машины. Анализ, синтез и применение. М.: Наука, 1974.
    a. Гэри М., Джонсон Д. Вычислительные машины и труднорешаемые задачи. М., 1982.
84. Колчин В.Ф., Севастьянов Б.А., Чистяков В.П. Случайные размещения. М.: Наука, 1976.
85. Рейнгольд Э., Нивергельт Ю., Део Н. Комбинаторные алгоритмы. Теория и практика. М.: Мир, 1980.
    a. Biham E., Shamir A. Differential Cryptanalysis of the Data Encryption Standard/ Springer-Verlag, 1993.
86. Biham E. Cryptanalysis of Multiple Modes of Operation/ Proc. ASIACRYPT'94, Springer- Verlag, 1994, c. 278-291.
87. Biham E., Knudsen L. Cryptanalysis of the ANSI X.9.52 CBCM Mode /Technion – Computer Science Department – Technical Report CS 0928, 1998.
88. Golic J. Intrinsic Statistical Weakness of Keystream generators //Advances in Cryptology – ASIACRYPT'94, Lecture Notes in Computer Science, v. 917, Springer- Verlag, 1995, pp. 91-103.
89. Gollmann D., Chambers W. Clock Controlled Shift Registers: a Review //IEEE J. Scl. Ar. Commun., 1989, v. 7, N 4, pp. 525-533.
90. Matsui M. Linear Cryptanalysis Method for DES Cipher//Pre-Proceedings of Eurocrypt'93, 1993, pp. 112-133.
    a. Meier W., Staffelbach O. Fast Correlation Attacks on Certain Stream Ciphers // J. Cryptology, v. 1, N 3, 1989, pp. 159-176.
91. Rueppel R. Good Stream Ciphers are Hard to Design. ICCST, Zurich, 1989, pp. 163-173.
92. Schnorr C. On the Construction of Random Number Generators and Random Function Generators//Advances in Cryptology – Eurocrypt'88, 1988, pp. 225-232.

Интернет-ресурсы.

http://www.fstec.ru/ - сайт Федеральной службы по техническому и экспортному контролю (ФСТЭК России);
www.fsb.ru – сайт Федеральной службы безопасности России (ФСБ России);
http://ess.ru/index.htm - сайт журнала «Специальная техника»;
http://www.st-s.su/index.htm - сайт журнала «Специальная техника и связь»;
http://www.inside-zi.ru/ - сайт журнала «Защита информации. Инсайд»;
http://www.pvti.ru/articles_14.htm - сайт журнала «Безопасность информационных технологий»;
http://kafedrasib.ru/?page_id=119 - сайт журнала «Информация и безопасность»;
http://www.sec.ru/ - портал по безопасности;
http://www.novocom.ru/ - сайт АОЗТ «Ново»;
http://www.mascom.ru/ - сайт ЦБИ «Маском»;
http://www.nelk.ru/node/9 - сайт НПЦ «Нелк»;
http://www.nppgamma.ru/- сайт ФГУП НПП «Гамма»;
http://www.infosec.ru/ - сайт НИП «Информзащита»;
http://www.bnti.ru/about.asp - Бюро научно-технической информации «Техника для спецслужб».

Руководитель курсовой работы

Доцент кафедры ИБ к.ф.-м.н., доцент ***Бутакова Н.Г.*** /_____ /

(подпись)

Задание получил

Студент учебной группы ИБ-31 ***Бройдо Г.Д.*** / _____ /

(подпись)

«___» октябрь 2025 года

АННОТАЦИЯ

Данная курсовая работа содержит:

- 53 страниц;
- 28 иллюстраций;
- 6 таблиц;
- 7 приложений;
- 4 использованных источников.

В данной курсовой работе объектами исследования были выбраны шифры: шифр Вернама, шифр Шеннона, ГОСТ 28147-89 (Режим Гаммирования).

Целью исследования в данной работе является сравнение шифром гаммирования на основе выбранных для рассмотрения в работе.

Для выполнения данной работы использовалась интерактивная среда разработки программного кода Visual Studio Code с применением высокоуровневого языка разработки Python.

В ходе выполнения работы были достигнуты следующие результаты:

- разработано необходимое для шифрования программное обеспечение;
- произведено шифрование отрывка литературного текста;
- произведен частотный анализ исходного текста и полученных шифртекстов;
- произведено сравнение выбранных шифров;
- сформулированы рекомендации по использованию данных шифров.

РЕФЕРАТ

Данная курсовая работа содержит:

- 53 страниц;
- 28 иллюстраций;
- 6 таблиц;
- 7 приложений;
- 4 использованных источников.

Ключевые слова: криптография, сравнение шифров, шифры гаммирования, шифр Вернама, шифр Шеннона, ГОСТ 28147-89 (Режим Гаммирования), частотный анализ.

Объектами исследования в данной работе являются шифры: шифр Вернама, шифр Шеннона, ГОСТ 28147-89 (Режим Гаммирования).

Цель исследования заключается в сравнительном анализе выбранных шифров, на основе изучения методов их криптоанализа и результатов шифрования отрывка литературного текста.

Параметрами для проведения сравнительного анализа являются:

- криптостойкость;
- удобство пользования (при ручной и программной реализации).

Для создания программ шифрования и расшифрования, а также подсчета биграмм и триграмм с последующим построением гистограмм встречаемости использовалась интерактивная среда разработки программного кода Visual Studio Code с применением высокоуровневого языка разработки Python.

В ходе выполнения работы были достигнуты следующие результаты:

- разработаны блок-схемы для выбранных алгоритмов шифрования и их расшифрования;
- разработано необходимое для шифрования программное обеспечение;
- произведено шифрование отрывка литературного текста;
- произведен частотный анализ символов, биграмм и триграмм в исходном тексте и в полученных шифртекстах;
- оценена криптостойкость выбранных алгоритмов;
- произведено сравнение шифров по указанным выше параметрам;
- сформулированы рекомендации по использованию данных шифров.

ПЕРЕЧЕНЬ УЛОВНЫХ ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ

ПО – программное обеспечение.

Криптография — область научных, прикладных, инженерно-технических исследований и практической деятельности, направленных на разработку средств криптографической защиты информации от несанкционированных пользователей.

Ключ — секретная информация, используемая криптографическим ким алгоритмом при шифровании/расшифровке сообщений, постановке и проверке цифровой подписи, вычислении кодов аутентичности (MAC).

Зашифрование – процесс зашифрования информации, то есть применения криптографического преобразования данных, эту информацию содержащих.

Расшифрование – преобразование шифрованного сообщения в подлежащую защите исходную информацию.

Открытый текст – оригинал сообщения.

Шифртекст (криптограмма) – результат процесса зашифрования.

Криптоанализ – наука о методах и способах вскрытия шифров.

НОД – наибольший общий делитель. Нотация: $\text{НОД} = (c, m)$ – наибольший общий делитель чисел c и m.

СОДЕРЖАНИЕ

ЗАДАНИЕ 2

АННОТАЦИЯ 7

РЕФЕРАТ 8

ПЕРЕЧЕНЬ УЛОВНЫХ ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ 9

СОДЕРЖАНИЕ 10

ВВЕДЕНИЕ 12

1 Анализ выбранных шифров 15

1.1 Анализ алгоритмов шифрования 16

1.1.1 Шифр Шеннона. Анализ алгоритма и построение блок-схемы 16

1.1.2 Шифр Вернама. Анализ алгоритма и построение блок-схемы 17

1.1.3 ГОСТ 28147-89 (режим гаммирования). Анализ алгоритма и построение блок-схемы 17

1.2 Историческое развитие исследуемых шифров 18

1.2.1 Шифр Вернама 18

1.2.2 Шифр Шеннона 19

1.2.3 ГОСТ 28147-89 19

1.3 Разработка ПО для шифрования 20

1.3.1 Разработка ПО для шифрования с использованием шифра Вернама 20

1.3.1.1 Разработка ПО для шифрования с использованием шифра Вернама (посимвольно) 20

1.3.1.2 Разработка ПО для шифрования с использованием шифра Вернама (побитово) 21

1.3.2 Разработка ПО для шифрования с использованием шифра Шеннона 22

1.3.2.1 Разработка ПО для шифрования с использованием шифра Шеннона (посимвольно) 22

1.3.2.2 Разработка ПО для шифрования с использованием шифра Шеннона (побитово) 23

1.3.3 Разработка ПО для шифрования с использованием ГОСТ 28147-89 (Режим Гаммирования) 24

1.3.4 Разработка ПО для использования всех выбранных шифров гаммирования 25

Вывод по главе 1 28

2 Сравнительный анализ криптостойкости шифров Вернама, Шеннона и ГОСТ 28147-89 (Режим Гаммирования) 29

2.1 Произведение шифрования с помощью разработанного ПО для исследуемых шифров 29

2.2 Построение гистограмм исходного и шифрованных текстов, биграмм и триграмм. Анализ результатов 32

2.2.1 Построение и анализ гистограмм относительных частот встречаемости букв в исходном тексте и зашифрованных текстах 32

2.2.3 Построение и анализ гистограмм относительных частот встречаемости триграмм в исходном тексте и зашифрованных текстах 44

2.3 Криптоанализ исследуемых шифров 49

2.3.1 Криптоанализ шифра Вернама и Шеннона 49

2.3.2 Криптоанализ ГОСТ 28147-89 (Режим Гаммирования) 50

2.4 Рекомендации по использованию исследованных шифров для защиты информации 50

Вывод по главе 2 51

ЗАКЛЮЧЕНИЕ 52

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 53

ПРИЛОЖЕНИЕ А 54

ПРИЛОЖЕНИЕ Б 55

ПРИЛОЖЕНИЕ В 56

ПРИЛОЖЕНИЕ Г 57

ПРИЛОЖЕНИЕ Д 58

ПРИЛОЖЕНИЕ Е 59

ПРИЛОЖЕНИЕ Ж 60

ВВЕДЕНИЕ

Одним из важнейших аспектов процесса передачи и хранения информации является обеспечение её безопасности. Данные должны сохранять свою целостность и недоступность для несанкционированных лиц вплоть до момента, пока сама информация остаётся актуальной [1].

Криптография — это область научных, прикладных, инженерно-технических исследований и практической деятельности, направленных на разработку средств криптографической защиты информации от несанкционированных пользователей.

Криптография как наука начала формироваться с началом распространения письменности, а некоторые криптосистемы появились еще в Древнем Риме. Особое развитие криптография получила во время первой и второй мировых войн в связи с необходимостью ускорения передачи информации без потери защищённости [2], а в послевоенное время развитие этой науки ускорилось благодаря широкому распространению средств электронно-вычислительной техники, сделавшей многие криптографические алгоритмы более нерелевантными.

В настоящее время криптографические методы защиты информации являются наиболее надежным, а в некоторых случаях, единственным средством противодействия различным угрозам. Их применяют как для защиты информации, обрабатываемой средствами электронно-вычислительной техники, так и для закрытия информации, передаваемой по каналам связи между различными элементами информационных систем [3].

Криптология — это обобщенная область защиты информации, которая подразделяется непосредственно на криптографию и криптоанализ.

Криптоанализ — это наука о методах и способах вскрытия шифров [3].

Основными задачами современной криптографии являются:

- обеспечение конфиденциальности информации;
- обеспечение целостности;
- обеспечение аутентификации;
- обеспечение невозможности отказа от авторства;
- обеспечение неотслеживаемости;
- решение проблемы управления ключами.

Шифры гаммирования размещаются в отдельном блоке в классификационной схеме шифров наряду с шифрами перестановок и шифрами замены. Процесс преобразования данных

выполняется посимвольно или побитово (XOR по модулю 2 или по выбранному модулю), с использованием шифрующей последовательности (гаммы) — последовательности случайных или псевдослучайных символов. Использование логической операции XOR обеспечивает тесную зависимость каждого зашифрованного бита или символа от соответствующего элемента гаммы. Такой подход позволяет сделать структуру шифртекста максимально приближенной к случайной последовательности, устраняя заметные закономерности, характерные для естественного языка.

В данной курсовой работе для выбранных шифров гаммирования будет проведен сравнительный анализ. Задача состоит в том, чтобы проанализировать криптостойкость всех шифров, и на основе результатов частотного анализа, а также гистограмм биграмм и триграмм, сделать вывод о криптостойкости и разработать рекомендации по их использованию в целях защиты информации.

Рассматриваемые в курсовой работе шифры все могут работать побитово с данными. Кроме того, шифр Вернама и шифр Шеннона может работать и посимвольно. Шифр Шеннона в оригинальном исполнении использует одноразовые гамма-блокноты, а сложение операндов может быть побитовым (XOR по модулю 2) или посимвольным. Успешное использование генераторов для получения гаммы любой желаемой длины (например, линейных конгруэнтных генераторов) приводит к тому, что гамма – уникальна для каждого элемента, на который она накладывается, а гистограмма частотного анализа шифртекста становится значительно более выровненной по сравнению с исходным текстом, что снижает вероятность вскрытия, баз знания ключа. Шифр Вернама использует ключ конечной длины, что может приводить к повторению гаммы, при шифровании текста большой длины (большей, чем длина ключа), что пагубно влияет на криптостойкость. Также в работе рассматривается шифр ГОСТ 28147-89, применяемый в режиме гаммирования. ГОСТ 28147-89 является российским криптографическим стандартом, который, однако, был заменен более современным и усовершенствованным его преемником – шифром «МАГМА», который в рамках данной курсовой работы не рассматривается.

Шифр Вернама – является классическим примером шифрования с одноразовыми ключами, обеспечивающим абсолютную криптографическую стойкость при соблюдения предоставляемых условии, которые будут рассмотрены в курсовой работе.

Шифр Шеннона развивает идею Вернама, предлагая концепцию псевдослучайной гаммы, генерируемой на основе ключа и математических преобразований, что делает алгоритм более применимым в реальных цифровых системах.

ГОСТ 28147-89 (режим гаммирования) представляет собой отечественный стандарт симметричного шифрования, в котором блочный алгоритм выполняет роль генератора

псевдослучайной гаммы, обеспечивая потоковое шифрование при сохранении криптостойкости базового стандарта.

В данной курсовой работе рассмотрены алгоритмы шифрования Вернама, Шеннона и ГОСТ 28147-89 (Режим Гаммирования), проведено шифрование литературного текста с использованием указанных алгоритмов, выполнен анализ их криптостойкости и разработаны рекомендации по их использованию.

1 Анализ выбранных шифров

Все выбранные шифры являются шифрами гаммирования, что распределяет их в один блок классификационной схемы шифров – блок шифров гаммирования.

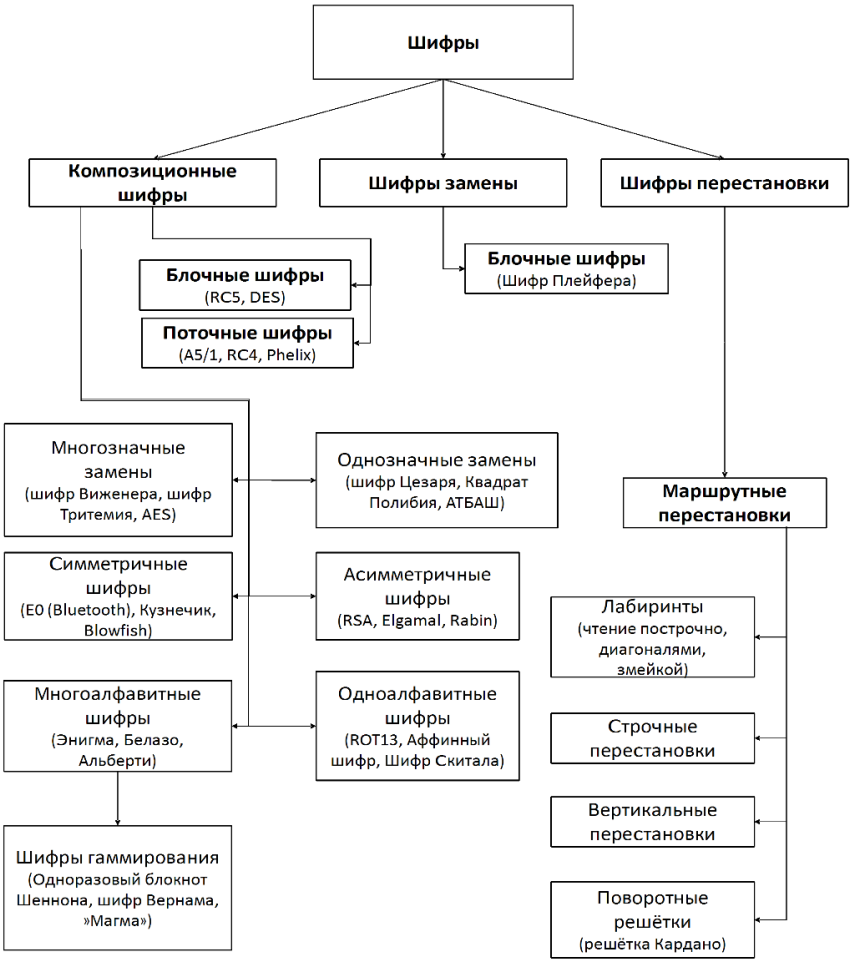


Рисунок 1 – Классификация шифров

Однако по их положению в дереве развития криптографии занимают разные места.

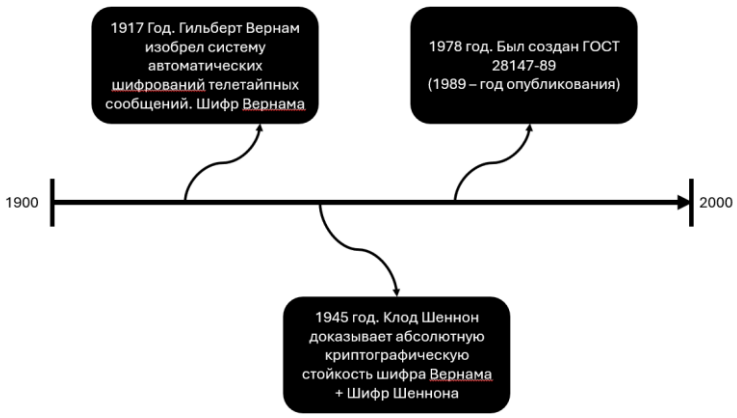


Рисунок 2 – Дерево развития криптографии

1.1 Анализ алгоритмов шифрования

1.1.1 Шифр Шеннона. Анализ алгоритма и построение блок-схемы

Шифр Шеннона в оригинальном исполнении использует одноразовые гамма-блокноты и стал активно использоваться в годы Второй мировой войны. Шифр Шеннона имеет разные реализации по модулю сложения операндов при шифровании: посимвольное или побитовое сложение.

Принцип работы:

1) Записываем Открытый текст в виде последовательности битов (или оставляем в символьном виде);
2) Записываем Шифрующую последовательность (гамму) в виде последовательности битов (или оставляем в символьном виде);
3) Складываем побитово (XOR по модулю 2) каждый бит Открытого текста и Шифрующей последовательности (или посимвольно складываем каждый символ по выбранному модулю – XOR по выбранному модулю);
4) Получаем Шифртекст.

Касаемо его надежности – в своей исторической работе «Communication Theory of Secrecy Systems» Клод Шеннон доказал абсолютную надежность как самого шифра Шеннона, так и шифра Вернама, при условии того, что гамма является случайной, выбирается равновероятно, не короче длины текста и выбирается новая гамма для каждого нового текста.

Для создания подходящей гаммы могут использоваться линейные конгруэнтные генераторы:

$$V_{n+1} = (aV_n + c)\ mod\ m, \tag{1}$$

где m – модуль, относительно которого вычисляет остаток от деления ($m \geq 2$);

a, c – константы ($0 \leq a < m; 0 \leq c < m$).

Однако с генераторами не все так просто. Для того, чтобы он дал гамму, которая будет иметь наибольший период равный модулю, необходимо учесть несколько требований к параметрам:

1) a – нечетное число;
2) c – нечетное число;
3) $a\ mod\ 4 = 1$;
4) $(c, m) = 1$;
5) $b = a - 1\ mod\ p = 0$, для каждого простого p, являющегося делителем m;
6) b кратно 4, если m кратно 4.

Получающаяся гамма позволяет получить шифротекст, гистограмма частотного анализа которого становится заметно более выровненной, по сравнению с гистограммой частотного анализа исходного текста, что значительно снижает вероятность найти связь исходного текста и шифротекста по совпадениям частоты встречи биграмм, не зная ключа, то есть повышает криптостойкость получаемого шифртекста.

Блок-схемы алгоритма шифрования и расшифрования (симметричный шифр, поэтому схема одна и та же) для побитового и посимвольного варианта представлены в приложении А и Б.

1.1.2 Шифр Вернама. Анализ алгоритма и построение блок-схемы

Шифр Вернама очень схож с шифром Шеннона, однако использует заранее определенную гамму (ключ) при шифровании. Шифр Шеннона тоже может использовать заранее определенную гамму – гамма-блокнот, однако тот пришел к использованию генераторов. Шифр Вернама использует всё те же гамма-блокноты или иные средства хранение информации о ключе (например – перфоленты).

Ключевой момент, который необходимо отметить при рассмотрении шифра Вернама – ограниченность гаммы. Для шифрования текста длиной, например, в 1000 символов, с абсолютной надежностью получаемого шифртекста – необходимо соблюсти требования, которые были описаны Клодом Шенноном (см. прошлый пункт). Как итог, если всё же ключ короче текста, то период ключа будет повторяться на всем тексте, что оставит уязвимость к взлому получаемого шифртекста.

Блок-схемы алгоритма шифрования и расшифрования (симметричный шифр, поэтому схема одна и та же) для побитового и посимвольного варианта представлены в приложении В и Г.

1.1.3 ГОСТ 28147-89 (режим гаммирования). Анализ алгоритма и построение блок-схемы

При работе ГОСТ 28147-89 в режиме гаммирования используется генератор для гаммы. Для работы этого генератора при шифровании, кроме самого исходного текста и ключа, размер которого 256 бит, поддается синхропосылка. Эта синхропосылка задает начальное состояние генератора. Делается это так:

1) Синхропосылка шифруется с использованием алгоритма простой замены (также из ГОСТ 28147-89). Результат записывается в 32-разрядные регистры N_3 и N_4 – младшие и старшие биты соответственно;
2) N_3 суммируется по модулю 2^{32} с константой $C_2 = 1010101_{16}$;
3) N_4 суммируется по модулю $2^{32}-1$ с константой $C_1 = 1010104_{16}$;
4) N_3 и N_4 переписываются соответственно в N_1 и N_2, которые затем шифруются с использованием алгоритма простой замены. Полученный результат является 64 битами гаммы;
5) Шаги 2-4 повторяются в соответствии с длиной шифруемого текста.

Сам исходный текст шифруется блоками по 64 бита – такими же блоками и генерируется гамма. Как было упомянуто, в генерации гаммы используется алгоритм простой замены. Он важен для работы данного режима работы, ведь он лежит в основе генерации гаммы. Построен алгоритм простой замены на сети Фейстеля, использует S-блоки для замены, а также циклические сдвиги.

Из особенностей этого шифра можно отметить то, что при изменении одного бита шифртекста меняется лишь один бит расшифрованного текста. С одной стороны – это помехоустойчивость, с другой – можно внести изменения в текст, даже не расшифровывая его.

Блок-схемы алгоритма шифрования и расшифрования (симметричный шифр, поэтому схема одна и та же) для режимов простой замены и гаммирования представлены в приложении Д и Е.

1.2 Историческое развитие исследуемых шифров

1.2.1 Шифр Вернама

Шифр Вернама был предложен в 1917 году телеграфистом Гилбертом Вернамом, который разрабатывал систему защиты телетайпных сообщений. Изначально ключевой материал представлял собой перфорированные ленты или бумажные таблицы с псевдослучайными символами, которые использовались синхронно на передающей и принимающей стороне и уничтожались после применения.

Вернам создал устройство, производящее указанные операции автоматически, без участия шифровальщика. Тем самым было положено начало так называемому «линейному шифрованию», когда шифрование/дешифрование является частью процесса передачи сообщения. До той поры шифрование было самостоятельной предварительной операцией, поэтому линейное шифрование существенно повышало оперативность связи.

Со временем идея была формализована как схема «одноразового блокнота», где гамма должна быть случайной, равной по длине сообщению и использоваться строго один раз. В 1940-е годы одноразовые блокноты активно применялись в военных и дипломатических системах связи разных государств, в том числе для защищённой переписки спецслужб.

1.2.2 Шифр Шеннона

Шифр Шеннона логически развивает идеи, заложенные еще в шифре Вернама. Сам Клод Шеннон стал родоначальником теории информации. Кроме этого, он написал труд по секретной связи, который послужил началом обширных исследований в теории кодирования и передачи информации. Шеннон определил основополагающие понятия теории криптографии, без которых криптография уже немыслима. Важной заслугой Шеннона являются исследования абсолютно криптостойких систем и доказательство их существования, а также существование криптостойких шифров и требуемые для этого условия. Шеннон также сформулировал основные требования, предъявляемые к надёжным шифрам. Он ввёл ставшие уже привычными понятия рассеивания и перемешивания, а также методы создания криптостойких систем шифрования на основе простых операций. [4]

По итогу развития вычислительных систем стали возможны реализации линейных конгруэнтных генераторов, которые и позволили соблюсти все требования, которые Клод Шеннон предъявил к шифрам для их криптостойкости.

1.2.3 ГОСТ 28147-89

История создания шифра и критерии разработчиков были впервые публично представлены в 2014 году руководителем группы разработчиков алгоритма Заботиным Иваном Александровичем на лекции, посвященной 25-летию принятия российского стандарта симметричного шифрования.

ГОСТ 28147-89 был утверждён в 1980-х годах как государственный стандарт СССР для криптографической защиты информации в системах обработки данных. Он разрабатывался как национальный симметричный алгоритм шифрования, призванный заменить зарубежные решения и обеспечить контроль за средствами защиты информации на уровне государства.

Первоначально ГОСТ 28147-89 определял общий алгоритм блочного шифрования, но в рамках стандарта были описаны и различные режимы его применения, включая режимы простой замены, гаммирования и гаммирования с обратной связью. Режим гаммирования появился как ответ на практические задачи потоковой обработки данных и защищённой

передачи информации по каналам связи, где важно шифровать последовательный поток без накопления данных в блоки.

Со временем на смену ГОСТ 28147-89 начали приходить новые отечественные стандарты, однако он до сих пор остаётся важным элементом криптографической инфраструктуры и исторически значимым алгоритмом. Идеи, заложенные в его конструкции и режимах работы, в том числе гаммирования, повлияли на разработку последующих российских криптографических стандартов.

1.3 Разработка ПО для шифрования

Для написания ПО мной был выбран язык Python 3. Для написания программного кода использовалась интерактивная среда разработки Visual Studio Code.

Во избежание потери символов при расшифровке текста, необходимо предварительно подготовить текст:

- Заменить все буквы «ё» (в любом регистре) на «е» в связи с тем, что шифрование происходит по алфавите мощности 32 (без буквы «ё»);
- Символы кроме точки «.», запятой «,» – убираются при шифровании, поэтому, если есть необходимость сохранить их, следует заменить такие символы на соответствующие слова (например: «–» заменить на «тире»).

1.3.1 Разработка ПО для шифрования с использованием шифра Вернама

1.3.1.1 Разработка ПО для шифрования с использованием шифра Вернама (посимвольно)

На вход программа для шифра Вернама с посимвольным шифрованием требует открытый текст, ключ и опциональный аргумент, который указывает – происходит посимвольное шифрование или расшифрование (в связи с особенностями работы с модулем по мощности алфавита).

Проведем проверку программы.

Возьмем первые три символа поговорки «Леопард не может изменить своих пятен.» и первые три символа слова «Криптография».

Получается, надо наложить гамму «КРИ» на текст «ЛЕО». Распишем индексы букв (счёт с нуля) в алфавите: Е=5, И=8, К=10, Л=11, О=14, Р=16. Л+К=11+10=21=Х; Е+Р=5+16=21=Х; О+И=14+8=22=Ц

Получили «ХХЦ». Сверим с программой:

```
##############################################################################
Проводим тест №8 (test_vernam_str)...

Исходный текст: Леопард не может изменить..., Длина: 35 символов
Зашифрованный            ХХЦЮТ ЮЗЭЕА ЦЕПВР ЦЮУРШ...
Расшифрованный           ЛЕОПА РДНЕМ ОЖЕТИ ЗМЕНИ...
Тест пройден успешно ✅

Тест №8 завершен!

##############################################################################
```

Рисунок 3 – Результат теста шифра Вернама (посимвольного)

Как мы видим на рисунке 3 с результатами теста – всё совпадает. Более того – программа по этому же ключу смогла расшифровать, а значит программная реализация верна.

1.3.1.2 Разработка ПО для шифрования с использованием шифра Вернама (побитово)

На вход программа для шифра Вернама с побитовым шифрованием требует открытый текст и ключ.

Проведем проверку программы.

Возьмем первые три символа поговорки «Леопард не может изменить своих пятен.» и первые три символа слова «Криптография».

Получается, надо наложить гамму «КРИ» на текст «ЛЕО». Распишем индексы букв (счёт с нуля) в алфавите: Е=5, И=8, К=10, Л=11, О=14, Р=16.

Теперь переведем эти значения в битовую репрезентацию:

$Е=5_{10}=00101_2$, $И=8_{10}=01000_2$, $К=10_{10}=01010_2$, $Л=11_{10}=01011_2$, $О=14_{10}=01110_2$, $Р=16_{10}=10000_2$.

$Л \oplus К=01011_2 \oplus 01010_2=00001_2=1_{10}=Б$;

$Е \oplus Р=00101_2 \oplus 10000_2=10101_2=21_{10}=Х$;

$О \oplus И=01110_2 \oplus 01000_2=00110_2=6_{10}=Ж$.

Получили «БХЖ». Сверим с программой:

```
##########################################################################
Проводим тест №9 (test_vernam_bin)...

Исходный текст: Леопард не может изменить..., Длина: 35 символов
Зашифрованный            БХЖАТ ЮЗЭЕШ ЖЩПВА ИЮЛОШ...
Расшифрованный           ЛЕОПА РДНЕМ ОЖЕТИ ЗМЕНИ...
Тест пройден успешно ✅

Тест №9 завершен!

##########################################################################
```

Рисунок 4 – Результат теста шифра Вернама (побитового)

Как мы видим на рисунке 4 с результатами теста – всё совпадает. Более того – программа по этому же ключу смогла расшифровать, а значит программная реализация верна.

1.3.2 Разработка ПО для шифрования с использованием шифра Шеннона

1.3.2.1 Разработка ПО для шифрования с использованием шифра Шеннона (посимвольно)

На вход программа для шифра Шеннона с посимвольным шифрованием требует открытый текст, параметры линейного конгруэнтного генератора и опциональный аргумент, который указывает – происходит посимвольное шифрование или расшифрование (в связи с особенностями работы с модулем по мощности алфавита).

Проведем проверку программы.

Возьмем первые три символа поговорки «Леопард не может изменить своих пятен.» и следующие параметры, согласно формуле (1): $V_0 = 2, a = 13, c = 7$.

Сгенерируем первые три значения (V_1, V_2, V_3)

$$V_1 = (aV_0 + c)\ mod\ m = (13 * 2 + 7)\ mod\ 32 = 1, \quad (2)$$

$$V_2 = (aV_1 + c)\ mod\ m = (13 * 1 + 7)\ mod\ 32 = 20, \quad (3)$$

$$V_3 = (aV_2 + c)\ mod\ m = (13 * 20 + 7)\ mod\ 32 = 11, \quad (4)$$

1=Б, 20=Ф, 11=Л. Получается, надо наложить гамму «БФЛ» на текст «ЛЕО». Распишем индексы букв (счёт с нуля) в алфавите: Б=1, Е=5, Л=11, О=14, Ф=20. Л+Б=11+1=12=М; Е+Ф=5+20=25=Щ; О+Л=14+11=25=Щ

Получили «МЩЩ». Сверим с программой:

```
##########################################################################
Проводим тест №6 (test_shenon_str)...

Исходный текст: Леопард не может изменить..., Длина: 35 символов
Зашифрованный            МЩЩЕЕ ШУЧОИ БДТВЯ ЩЭЙИО...
Расшифрованный           ЛЕОПА РДНЕМ ОЖЕТИ ЗМЕНИ...
Тест пройден успешно ✅

Тест №6 завершен!

##########################################################################
```

Рисунок 5 – Результат теста шифра Шеннона (посимвольного)

Как мы видим на рисунке 5 с результатами теста – всё совпадает. Более того – программа по этому же ключу смогла расшифровать, а значит программная реализация верна.

1.3.2.2 Разработка ПО для шифрования с использованием шифра Шеннона (побитово)

На вход программа для шифра Шеннона с посимвольным шифрованием требует открытый текст, параметры линейного конгруэнтного генератора.

Проведем проверку программы.

Возьмем первые три символа поговорки «Леопард не может изменить своих пятен.» и следующие параметры, согласно формуле (1): $V_0 = 2, a = 13, c = 7$.

Первые три значения (V_1, V_2, V_3) мы уже сгенерировали в пункте 1.3.2.2.

Получается, надо наложить гамму «БФЛ» на текст «ЛЕО». Распишем индексы букв (счёт с нуля) в алфавите: Б=1, Е=5, Л=11, О=14, Ф=20.

Теперь переведем эти значения в битовую репрезентацию:

Б=1_{10}=00001_2, Е=5_{10}=00101_2, Л=11_{10}=01011_2, О=14_{10}=01110_2, Ф=20_{10}=10100_2.

Л⊕Б=01011_2⊕00001_2=01010_2=10_{10}=К;

Е⊕Ф=00101_2⊕10100_2=10001_2=17_{10}=С;

О⊕Л=01110_2⊕01011_2=00101_2=5_{10}=Е.

Получили «КСЕ». Сверим с программой:

```
##########################################################################
Проводим тест №7 (test_shenon_bin)...

Исходный текст: Леопард не может изменить..., Длина: 35 символов
Зашифрованный            КСЕЩЕ ШЛЗМР ЭШИВЯ ХЭБЦО...
Расшифрованный           ЛЕОПА РДНЕМ ОЖЕТИ ЗМЕНИ...
Тест пройден успешно ✅

Тест №7 завершен!

##########################################################################
```

Рисунок 6 – Результат теста шифра Шеннона (посимвольного)

Как мы видим на рисунке 6 с результатами теста – всё совпадает. Более того – программа по этому же ключу смогла расшифровать, а значит программная реализация верна.

1.3.3 Разработка ПО для шифрования с использованием ГОСТ 28147-89 (Режим Гаммирования)

На вход программа для ГОСТ 28147-89 в режиме гаммирования требует открытый текст, 256-битный ключ, синхропосылку размером 64 бит.

Проведем проверку программы. В связи со сложностью и трудозатратностью реализации алгоритма на бумаге, за эталон брать нечего. Касаемо ГОСТов – имеются лишь проверки для режима простой замены. Однако в рамках реализованного алгоритма, он не проходит тестовые примеры из ГОСТ 34.13 – 2015 для простой замены (вероятно, связано с особенностями того, как обрабатываются биты перед каждым раундом). Среди множества найденных мною статей пользовательских реализаций – я не нашел различий с написанным мною алгоритмом, однако примеры из ГОСТ не подтвердились. Тем не менее – алгоритм действительно генерирует случайную (псевдослучайную) гамму, которая придаёт итоговому шифртексту криптографическую стойкость.

```
#########################################################################
Проводим тест №4 (test_gamming_2)...

Исходный текст: Меня зовут Вито Скалетта. Я..., Длина: 1051 символов
Биты из исходного текста: 01100 00101 01101 11111...
Зашифрованный             10110 01010 00100 11000...
Зашифрованный (буквами)   ЦКДШИ ХХЪКВ ЕЧНЗК ЧШЫАХ...
Расшифрованный            01100 00101 01101 11111...
Расшифрованный (буквами)  МЕНЯЗ ОВУТВ ИТОСК АЛЕТТ...
Тест пройден успешно ✅

Тест №4 завершен!

#########################################################################
Проводим тест №5 (test_simple_sub_2)...

Исходный текст: Меня зовут Вито Скалетта. Я..., Длина: 1051 символов
Зашифрованный             10110 00110 10001 11111...
Зашифрованный (буквами)   ЦЖСЯГ ФПЛОЩ ЯЮКЦФ ШНЖОЭ...
Расшифрованный            01100 00101 01101 11111...
Расшифрованный (буквами)  МЕНЯЗ ОВУТВ ИТОСК АЛЕТТ...
Тест пройден успешно ✅

Тест №5 завершен!

#########################################################################
```

Рисунок 7 – Результат теста ГОСТ 28147-89 при режиме гаммирования и простой замены

Как видно из результатов тестирования – алгоритм действительно шифрует и расшифровывает сообщение.

1.3.4 Разработка ПО для использования всех выбранных шифров гаммирования

В ходе разработки была также написана консольная утилита для использования всех алгоритмов, описанных в курсовой работе.

Основная функция, ответственная за выбор нужного шифра или вызова тестов шифров представлена на рисунке 8.

Вид самого меню, вызываемого консольной утилитой представлен на рисунках 9-13.

gamming_ciphers.py × hist_analysis.py hists_cw.py

gamming_ciphers.py > ...

```python

def main():
    option: int = -1
    while (True):
        try:
            clear_screen_ansi()
        except Exception:
            try:
                clear_terminal()
            except Exception:
                return
        print("""Шифрование шифрами гаммирования
    Выберите:
    0) Закрыть программу...
    1) Использовать Шифр Шеннона
    2) Использовать Шифр Вернама
    3) Использовать Шифр Гаммирования по ГОСТ 28147-89
    4) Запустить тесты""")

        try:
            option = int(input()[:1])
        except ValueError:
            continue
        try:
            clear_screen_ansi()
        except Exception:
            try:
                clear_terminal()
            except Exception:
                return
        if option == 0: break
        else:
            if option == 1:   menu_shenon()
            elif option == 2: menu_vernam()
            elif option == 3: menu_gost()
            elif option == 4: menu_tests()
```

Рисунок 8 – Основная функция меню выбора шифров

Как видно из результатов тестирования

```
Шифрование шифрами гаммирования
    Выберите:
    0) Закрыть программу...
    1) Использовать Шифр Шеннона
    2) Использовать Шифр Вернама
    3) Использовать Шифр Гаммирования по ГОСТ 28147-89
    4) Запустить тесты
```

Рисунок 9 – Меню выбора шифра

```
Шифрование шифром Шеннона
    Выберите:
    0) Выйти назад...
    1) Зашифровать сообщение  (символьное)
    2) Расшифровать сообщение (символьное)
    3) Зашифровать сообщение  (бинарное)
    4) Расшифровать сообщение (бинарное)
```

Рисунок 10 – Меню шифра Шеннона

```
Шифрование шифром Вернама
    Выберите:
    0) Выйти назад...
    1) Зашифровать сообщение  (символьное)
    2) Расшифровать сообщение (символьное)
    3) Зашифровать сообщение  (бинарное)
    4) Расшифровать сообщение (бинарное)
```

Рисунок 11 – Меню шифра Вернама

```
Шифрование шифром ГОСТ 28147-89
    Выберите:
    0) Выйти назад...
    1) Зашифровать сообщение  (символьное)
    2) Расшифровать сообщение (символьное)
    3) Зашифровать сообщение  (бинарное)
    4) Расшифровать сообщение (бинарное)
```

Рисунок 12 – Меню ГОСТ 28147-89

```
Тест №12 завершен!

##############################################################################
Проводим тест №13 (test_long_text_vernam_bin)...

Исходный текст: Меня зовут Вито Скалетта. Я..., Длина: 1051 символов
Зашифрованный            ЖХЕРХ АБГТЦ АНДБВ ПЩЛСВ...
Расшифрованный           МЕНЯЗ ОВУТВ ИТОСК АЛЕТТ...
Тест пройден успешно ✅

Тест №13 завершен!

##############################################################################
Итоговый успех тестирования: 13/13 = 100.0 %
Press any key to continue . . .
```

Рисунок 13 – Вывод опции «Запустить Тесты»

Функция «Запустить Тесты» проводит тестирования, что по итогу расшифрования зашифрованного текста одним и тем же ключом выйдет один и тот же исходный текст.

Вывод по главе 1

В данной главе были рассмотрены шифры гаммирования: шифр Вернама, шифр Шеннона, ГОСТ 28147-89.

Разработав алгоритмы шифров, стало заметно, что они имеют в себе много общего, однако и отличий среди них хватает. Шифр Вернама базово гаммирует текст, используя ключ конечной длины. Шифр Шеннона развивает идею шифра Вернама и идеи Клода Шеннона по реализации криптографически стойких шифров, используя линейные конгруэнтные генераторы для генерации гаммы необходимой длины. ГОСТ 28147-89 же использует более сложную схему с сетью Фейстеля с 32 раундами, f-функцией преобразования, нелинейными преобразованиями (заменами), а также битовые сдвиги.

В результате работы стало понятно, что шифр Вернама в современном мире имеет скорее историческую ценность, а шифр Шеннона и ГОСТ 28147-89 в свою очередь могут применяться и по сей день, с поправкой на их устаревание в связи с колоссальным развитием вычислительных мощностей современных электронно-вычислительных машин. При использовании таких шифров важно понимать, к насколько важной и ценной информации они применяются, а также насколько важна безопасность этой информации.

Были разработаны программы для шифрования текстов при помощи указанных выше алгоритмов. Основным препятствием стала труднореализуемость ГОСТ 28147-89 на бумаге, что делает практически невозможным поиск ошибки в реализации данного шифра.

2 Сравнительный анализ криптостойкости шифров Вернама, Шеннона и ГОСТ 28147-89 (Режим Гаммирования)

2.1 Произведение шифрования с помощью разработанного ПО для исследуемых шифров

Для шифрования мной был выбран монолог главного героя пролога сюжета из компьютерной видеоигры «Mafia II» Вито Скалетта: «Меня зовут Вито Скалетта. Я родился на Сицилии, в 25-м году. Этот малыш — я. Стою с родителями и сестрой Франческой, у нашего старого дома. Я мало что помню о тех временах, разве что жизнь была тяжёлой, а потом отец решил, что нам пора уезжать. Покинуть Сицилию. Пересечь океан и начать новую жизнь в Америке. Никогда в жизни я не видел ничего подобного Эмпайр-Бэй. Он был прекрасен. Впрочем, трудно представить нечто более скверное и отвратительное, чем наша новая конура. «Американская мечта»… Скорее уж кошмар. Отец стал работать в порту на человека, который перевёз нас сюда. Он работал как проклятый, зарабатывал гроши и почти всё спускал в кабаке. Со временем меня отдали в школу. Мне нужен был английский, но чёрта с два я бы заговорил на нём с итальянцами. Там я и встретил Джо. Постепенно мы стали лучшими друзьями. У обоих свистело в кармане, а пристойной работы не было - и мы затеяли своё скромное дело. Что ж, тогда нам не повезло. На дворе стоял 43-й год. Шла война, и для высадки на Сицилии, нужны были солдаты, говорящие по-итальянски. Мне было восемнадцать, и в тюрьму я не хотел. Кто сказал, что нельзя вернуться на Родину?..».

При рассмотрении как исходного, так и зашифрованного текста, будет использоваться 32-буквенный алфавит (без буквы «Ё»).

Убрав все символы, кроме алфавитных, точек и запятых, я получил исправленный текст, который уже и был аргументом в программе. Однако для самих шифров он всё ещё не подходит, так как самим алгоритмам нужен текст лишь из букв (без пробелов, точек и запятых), поэтому посредством реализованной функции был получен следующий открытый текст:

«менязовутвитоскалеттатчкяродилсянасицилииивдвадцатьпятомгодутчкэтотмалышятчкстоюсродителямиисестройфранческойунашегостарогодоматчкямалочтопомнюотехвременахтчктчктчкразвечтожизньбылатяжелойтчкапотомотецрешилзптчтонампорауезжатьтчкпокинуть сицилиютчкпересечьокеаниначатьновуюжизньвамерикетчкникогдавжизнияневиделничегозптподобногоэмпайрбэйтчконбылпрекрасентчквпрочемзпттруднопредставитьнечтоболеескверноеиотвратительноезптчемнашановаяконуратчкамериканскаямечтатчкскорееужкошмартчкотецсталработатьвпортуучеловеказпткоторыйперевезнассюдатчконработалкакпроклятыйзптзарабатывалгрошиипочтивсеспускалвкабакетчксовременемменяотдалившколутчкмненуженбылангл

ийскийзптночертасдваябызаговорилнанемсительянцамитчктамяивстретилджотчктчктчкпосте пенномысталилучшимидрузьямитчкуобоихсвистеловкарманезптапристойнойработынебылои мызатеялисвоескромноеделотчктчктчкчтожзпттогданамнеповезлотчкнадворестоялсороктрети йгодтчкшлавойназптидлявысадкинасицилиинужныбылисолдатызптговорящиепоитальянскит чкмнебыловосемнадцатьзптивтюрьмуянехотелтчкктосказалзптчтонельзявернутьсянародинут чк».

Зашифруем текст при помощи разработанной программы шифром Вернама (посимвольно). Используем тот же ключ – «КРИПТОГРАФИЯ».

После обработки получим следующий шифртекст:

цххощ ьегтц рсшбт пэухв ажяйй ацуъщ фпнфщ зашуч ързта шюяьм чодьп уошыс бъеба апрлп аюьзт адьбб рвмзь хуоюц лбееъ пшщья тыъхс юциээ изчсс бтфшн нюмэю охзку фяхюя баэсь нтцсп екячъ иэайъ цфвящ денаа ыкдбв цхъхр мбпуя ьпофэ ьмвчю иошвц ыааиж рщазх ччбйа сэаач нърыф щфгвь жяйщю тчябх мсьюз хшжбй штхрщ щдбмц щчорш нфяяь мхэфб бциых ымрфф вцнхт лтмтъ цтцое циыхз йэнсъ тиынь яднюп юдэсф оххнн юеыбо мабсс сбъць уйояр щтпкб ньден тпдцц пьпюд аугдб цоъхм адоеш трхдб вцращ ихсюк дъэцф ъьхтр фъзьх уляьи чпжяд цэизт ыстау тнчгш пденр мщшзф рхаьо вьелъ яьзта ььухе зойши фпваъ ъожнх ывиъв одютф ъымяц ядбцз еяцбп ъицба нютвш ъуяня чричн фщриф ибйшс эрфйн ьрущт штаою уюьлс цбакр рфйяь лкпэс уюшьр ошзъч фяибп зщйкы кщтпг ъежяй ыюкяч ъиэеа фдчпц бцоош вмтнх гъжьъ рхнзо дчсгъ тыжыи эщйтщ пюдыс зедъя ыфкпс пючач цбшар ъяорх мерск ыдояд гьижя йьрфо ърфвр щъзхф оэден вчюъц фяцад утхнб цлебъ пэцог чмрлт фшвщк вьижя йэюйэ ъгфти еъдхю кщтюп рнщпо ьрчяъ яхюйб циърй эдйрх бпунт ьгцта ипльщ бшхщщ вьпэо щмдхю ъжьаъ ътлтц ьюоцб ахюгш имкьх фбьех зяцсб ъхпцр саееъ нйыщэ вьнвр щъзуу цуден илфкн уэицб алфлу къырм щъыгб икркт шхвшы юсыяр ршымп дйкят чцбша зиъут юижик жпхаь цхзка хдллу эфьфх мбига рълщэ хшвжж пжьыо яушют щусбъ тбаян рзфуж щвяба ыиыьы збпах вдкфп нфшно шхвде н

Рассмотрим шифртекст, полученный при помощи шифра Вернама с побитовой гаммой (ключ тот же):

жхерх абгтц андбв пщлсв ажяхх ажлъе тпнфщ чьшгз ъмзта рюяшм зраап уорын эъхэь ьпрлп рашзв юааэб ръмчш хгрюж лбееъ пдщья тгфхс южцщэ иччнн бтфшс йюмбю осзкл дябюя эьбнь нкжнп екячв жэабъ иавяе ащйаа укъэв жйъйо мбпгя шпокщ аквчю ирдвж гььжж рсрчб чзэеь нэашз сърык хигвь жяхею взяэс мсьюч бшцэе дмхрс щъэмж ечоош нфяяш мебрэ эциуе гирдк вжйхт гвтвъ жмцоб циуеч хэннъ кжынь яъйюп аабнф охесй юхгэо кабйб нэъжв ухияр свпкб нващй тпджи пьпаа ьугдщ жръхм юаобш тиеъэ вжоье жхсюк ъъэжк ъастр фъчшх гуяаж чпжяъ жэичт гнтал всзгш пащйр мсшча реюшо ььегъ яшзвю шаухе зохди дпвьф ъожнй ывидв овютф ъгияж яаэрз еяжэп ъииэь йютъш дгяня чмжчн фщофф иэедн эрфйс шргет дмаою

гашлб иэьдр рфйяш лкпщн уюшьа рдзъз ряжбп зщхкы кетпг ъежях ыюкяч вжэеш дъзпж эцоиш вмвсб гъшшв охнзо ъзсуд тгаыи эщхвщ паагн зедъя ыфкпн пшчач жэдаа дяоох меанк ыфряш гьижя хшрдр ъмтвр съчбф обащй вчюъи аяжюа лмхнщ жусбъ пщжиг чмаув фшьхт ььижя хщюйб ъытти еъъбю кетюп рнспр шрзяъ ясюйщ жцърй бахох бпгсв ьуить жпльщ эдхще вапэо смъбю ъшшьф ътгви шюоиэ ьсюгр иткье кэабх зяжнэ ъепцм наееъ схыщб вайвр съчгу жлащй илфкс гэииэ ьлфлл кдырм еъггб ивафв шеьфг шсыяа одымп ахдят чжэда чцълм юижиф цпеюш жсзкш еъллг братх мщиыь ръухб сшвжц пцьыр ялцют сгнэъ вэьяй рзфгш евяэь гжыьу чэпае ьаттп нфшсо шеьащ й

Рассмотрим шифртекст, полученный при помощи шифра Шеннона с посимвольной гаммой:

нщшхм цсэыю ырыбб тьйнш хкцдш ьстел шбофь юырът сючан фнтга кезжл эзрца фкдфп жчцрг зйыуэ пяюхг бтяоз экщеф лявсщ тпэяж ехжпъ жбзаэ чкцзй чзшпи ьъзьй ащщлу чцрцж ьчлбк ъоедц щэцъд дзщбх езтюм улхже пспао бдхчд отяжж зчеяд ъмафк зспжщ вуъфа щблжш чждиц йухдо иймцу джзфэ жвафц щтцпе ъюшнъ ьмщшм вояйс фуфьх мжфшю тижкы шампг хшщли вчшля йфнуп чспйю щщйис рсгаж мэьфщ иасва цяьък эппхж пхшди цмцшь ьоона диойу цукйй снюэс мпулх шфшэб оиъня взехс йхеэг ллмец пьфзш жщчуу фпъжх гээеч щтние шсвлс окком йржвы схпвй йбанп баючл жзпйъ есуцз афулф квкяб кыуэп чюзчц чбргр лъйюъ шлтмш тжлбх иршыь еъпяе вгчоэ ъгнью цгхмт срувы сочфъ оюшеъ ригпи ышмвн зймдь патмб юъжут ъйычь еьвют бдьжз убъдп уьхип сплэю зщфыш пдины щьсеы ръкцд къеюв ммпжа чытзэ ьньюж пибаь чнэяд мяжяй укбвн ббобн сафсе ъняэе йцфнж жьбъш нюхэг хдпду бтиьп хнырн ыусюъ ытэкц длмпн евшфс щэюрм хшыуэ рдъйй ыуйчз эояжщ съшсщ вмьцй ьачцс агсфм цющцт рижаи ббфре сщзмв наешы кцбън яржвщ хнфюе нгймд ьпыфз впцдн фксйо шэшши фяймр ежлзз сзуио щщлжв ачящб ыкщеь вйафи ыухдм яиъеу длхфш юшцйк эъоне ммыиа бтеше эсввп стпчс ъмфнд охпсш оывшп щнвдя рэеял бвлще ифхзб жчауч ычзчю ялюбг зйиги юубст ьетзн эушзп рббуи аголв аутмл беяюп мхкдп щкгжй жбфвй цбимя хвдио ешгйй ъамох мтюфп брббп омома сямис юдлфщ амнлг н

Рассмотрим шифртекст, полученный при помощи шифра Шеннона с побитовой гаммой:

нсжйв жнщыю ымгбэ тъбйф хкирж ьнкхл цэмфъ юуадв бючьн фбтгш фщзцу щчирь ккърп жзцоу чхылщ пяюйг бкяоз эфехд ляшсх тпэяж еешпш цэчюэ чкфзх чзшпф ъвзас аххлл зцожш шзуэт аощаф сщцрф ъчщщс щчтри угбже пнпюо эшечъ орярж ззщят вкькк знпже ълъкь щщлцж чшажц хлхфр фймрл ъжзрэ жьькж евдпб вьшбъ ъмефв тряйй тлкьй идфжю иишкы аюрпг йфщгц ъчшуя йдйлп чнпйю ехбин мбыюц тэтрщ аюншр ияъвд бппйж пхжшж жтжжь ъомню ажохл фгфхй ййюэс впугб фкшбэ мрфся взбхй хйеэы ллмбж пькзц жечлг кпшцс ыээщч щкйде шнтлй итров еожьу йептй сэьнп эььчл жзпхъ хйужч акулф фъаяэ кылщп зюзчф чэмыа уъйюф дутвф ржлэх иодыь бвпящ вгчис русшь жгйтт нмуъы нмзкъ мюцща ржгпа ыфвтс чймва

памиб юджлв дхызъ щвврт бдъжз гэътп уаеип нпгщю зщкыж пвфны хшсеы мркир ивбюш мвпдш зуичб шньшц пиэьъ чйсяф тяфяе лрбьй бщиэн бюабх фсяэщ еффйж дьэъж ншйэг йапдг эиивп енымн ылньт ыкэки рлмпс хвцрс сщюом йдылщ мъъее ылхчз эряфб нвжсх вкьае трзжб шгнкм иющцм мжцюф сщтмх схзкъ йьешг кдщфс яршвщ хйиьх суймв апыкз апашн дфнйо цбжшж рябкм ецучч йзлцо ххлжь ьчяеэ ытхщв веьта ылхфт яцвбл ълйрц южцбк бъмнб ртыжь бксфе энтрп нкпчн ъкфйш мепнж оыъжп хйадя мэхял сълех иксзщ рчюгз ызччю ялрэг зхдыи агсйм ахтзй элжзп аээгр юымль ьуккг щхяюп мстъп хкгжх жщдьх дщжря хьажо сфыйх ъюмий ттррп щоэщп риммю нямжн ьдлис ртйлы й

Рассмотрим шифртекст, полученный при помощи ГОСТ 28147-89 с ключом «0xfe09ed8c776702696b9818a2f959bb75664f4ce67f3913bd4d8ce5fd9e57a93d» и синхропосылкой «0xbcff8a402c7d467d»:

ббнцп еылнй пэфщи цксгы акпжй ссним дмюдц йяулх стлтд дгфхк ыетбм двчдк кйжйд оцвыс ьйлдд джпзю рхюхц ядмбй ьтдяш ътьыы эгиор лйсшн ъщаэо ньлкщ зцккщ жрюос цацкш ккшбы гуьре шмшпл тглбз ъбнкь мвушр иышмб якыхо шчцел ъхьца пмонл тзцэв йшхцу ьфпык каиыз щыюви лчтщщ чдбэф нволы тгучи вабаэ ывдщи яьсвб щцщгр рбугп мвщпз пщфьи пъъйй ъллию мфйзб хэхпж игмдц туяов уушйх зсаыт мццжо шлжщт йгърс иадем ъбббю оьздэ злдуж зсхвз гюшов жбанх ехнно пбятж эптьр вусмц анящу ясуял ьучмь амебе асщкб наечр зншвщ смгхб хысхк шуыгй юьшыш ъчггэ джшнз ачщео ужоди охсох куймо фампн юткмф щшьнг ббсиэ сфнкз чюшею пщуйй зыеиз бйуса дсгцз гелди ъьчыг фдчсн ошвюи пбйлщ бшбхр кицха хжгкс ахэдо себез хиарт ядпыр яфкхь яабйм сейкч щмцаю рьуйю цхбхп дкыкт бынъщ ъккъь мухфг ецбмх еъвйч дпымр шшмпг пяльж хшхум шякящ шющьл щкюзб чълвф овпуя тящип тчуцш вуесн тдфпм вкйщп злчбх брцжа хюпжс чзбфэ ежбпж зрвня утээс фловн ныбым сймоз кмшрш ишщкц брзфг щьткш мийым бйзсы нззяе ьъсгщ меынц щеэвг юъйюп эоэфщ йкэлр офнаю ашзще фъъцц мчьфа ваюхб ексфп уфжхп эълцх свяюя щьйяъ сшичк осчхв рючфэ ъцъчл ебаын юкнбф зтизф трсхг цклэс яхгчэ юыешя пщмси шэщчю ньегй ююсны югшвш ишдхч ьогмз фчьгэ лйхяы бкбгс ерйео иинжх аушюв эгэшй ромпя щрнзи нюмьв емкеы жгцлк ецщеб ишйъл овичл мвкки мпцфх ткчщ ж

2.2 Построение гистограмм исходного и шифрованных текстов, биграмм и триграмм. Анализ результатов

2.2.1 Построение и анализ гистограмм относительных частот встречаемости букв в исходном тексте и зашифрованных текстах

Рассмотрим частоту встречаемости символов открытого (преобразованного) текста. Всего в имеющемся тексте 1051 символ, частота встречаемости каждого из них приведена в таблице 1:

Таблица 1 – частота встречаемости букв открытого текста

Буква	А	Б	В	Г	Д	Е	Ж	З	И	Й	К
Частота встречаемости, раз	81	16	37	12	25	76	10	27	60	13	62
Относительная частота встречаемости, %	7.71	1.52	3.52	1.14	2.38	7.23	0.95	2.57	5.71	1.24	5.9
Буква	Л	М	Н	О	П	Р	С	Т	У	Ф	Х
Частота встречаемости, раз	45	34	47	97	34	47	43	110	21	1	4
Относительная частота встречаемости, %	4.28	3.24	5.8	9.23	3.24	4.47	4.09	10.47	2	0.1	0.38
Буква	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я	
Частота встречаемости, раз	8	49	9	1	0	17	17	3	6	25	
Относительная частота встречаемости, %	0.76	4.66	0.86	0.1	0	1.62	1.62	0.29	0.57	2.38	

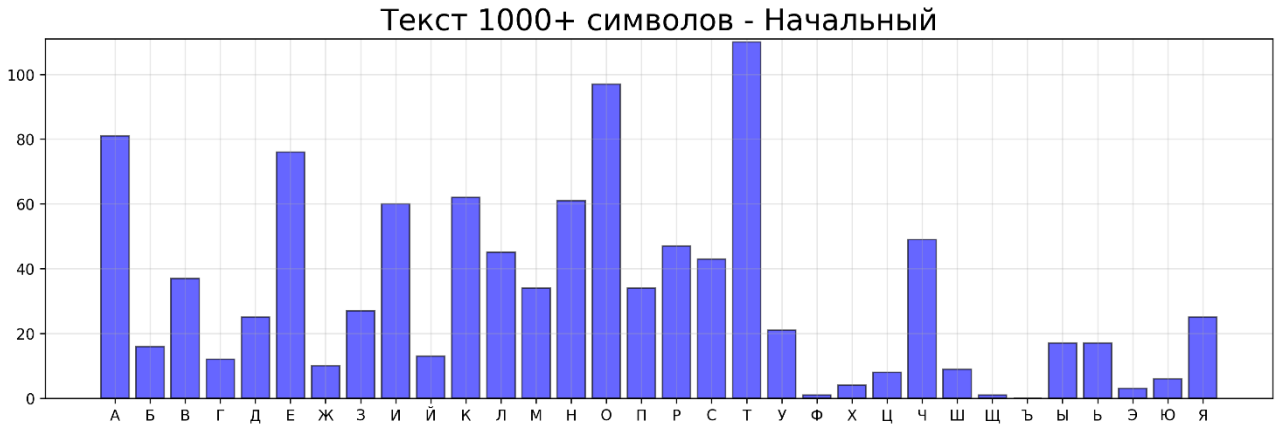


Рисунок 14 – Гистограмма встречаемости букв открытого текста

Заметно, что на гистограмме есть четко выраженные пики на буквах «А», «Е», «О», «Т». А есть буквы, которые встречали считанные разы – «Ф», «Щ», «Ъ», «Э». В общем – гистограмма выглядит достаточно неоднородной с пиками и просадками.

Теперь рассмотрим частоту встречаемости букв в тексте, зашифрованном при помощи шифра Вернама (посимвольного).

Таблица 2 – частота встречаемости букв в тексте, зашифрованном при помощи шифра Вернама (посимвольного)

Буква	А	Б	В	Г	Д	Е	Ж	З	И	Й	К
Частота встречаемости, раз	45	46	27	14	34	26	18	26	34	23	23
Относительная частота встречаемости, %	4.28	4.38	2.57	1.33	3.24	2.47	1.71	2.47	3.24	2.19	2.19
Буква	Л	М	Н	О	П	Р	С	Т	У	Ф	Х
Частота встречаемости, раз	17	22	36	33	37	43	27	43	28	39	49
Относительная частота встречаемости, %	1.62	2.09	3.43	3.14	3.52	4.09	2.57	4.09	2.66	3.71	4.66

Продолжение таблицы 2

Буква	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я	
Частота встречаемости, раз	44	24	32	34	50	30	46	27	34	40	
Относительная частота встречаемости, %	4.19	2.28	3.04	3.24	4.76	2.85	4.38	2.57	3.24	3.81	

Рисунок 15 – Гистограмма встречаемости букв в тексте, зашифрованном при помощи шифра Вернама (посимвольного)

Заметно, что на гистограмме нет четко выраженных пиков. В общем – гистограмма выглядит достаточно однородной, лишь с малыми пиками и просадками.

Теперь рассмотрим частоту встречаемости букв в тексте, зашифрованном при помощи шифра Вернама (побитового).

Таблица 3 – частота встречаемости букв в тексте, зашифрованном при помощи шифра Вернама (побитового)

Буква	А	Б	В	Г	Д	Е	Ж	З	И	Й	К
Частота встречаемости, раз	29	33	31	24	23	29	51	33	28	45	38
Относительная частота встречаемости, %	2.76	3.14	2.95	2.28	2.19	2.76	4.85	3.14	2.66	4.28	3.62
Буква	Л	М	Н	О	П	Р	С	Т	У	Ф	Х
Частота встречаемости, раз	38	36	33	17	45	40	30	34	19	38	44
Относительная частота встречаемости, %	3.62	3.43	3.14	1.62	4.28	3.81	2.85	3.24	1.81	3.62	4.19
Буква	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я	
Частота встречаемости, раз	23	30	30	37	34	31	34	37	30	27	
Относительная частота встречаемости, %	2.19	2.85	2.85	3.52	3.24	2.95	3.24	3.52	2.85	2.57	

Рисунок 15 – Гистограмма встречаемости букв в тексте, зашифрованном при помощи шифра Вернама (побитового)

Заметно, что на гистограмме нет четко выраженных пиков. В общем – гистограмма выглядит достаточно однородной, лишь с малыми пиками и просадками.

Теперь рассмотрим частоту встречаемости букв в тексте, зашифрованном при помощи шифра Шеннона (посимвольного).

Таблица 4 – частота встречаемости букв в тексте, зашифрованном при помощи шифра Шеннона (посимвольного)

Буква	А	Б	В	Г	Д	Е	Ж	З	И	Й	К
Частота встречаемости, раз	34	42	32	21	32	38	38	33	32	37	25
Относительная частота встречаемости, %	3.24	4	3.04	2	3.04	3.62	3.62	3.14	3.04	3.52	2.38
Буква	Л	М	Н	О	П	Р	С	Т	У	Ф	Х
Частота встречаемости, раз	30	41	35	28	45	23	40	27	33	37	32
Относительная частота встречаемости, %	2.85	3.9	3.33	2.66	4.28	2.19	3.81	2.57	3.14	3.52	3.04
Буква	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я	
Частота встречаемости, раз	32	30	39	33	34	28	31	31	31	27	
Относительная частота встречаемости, %	3.04	2.85	3.71	3.14	3.24	2.66	2.95	2.95	2.95	2.57	

Рисунок 15 – Гистограмма встречаемости букв в тексте, зашифрованном при помощи шифра Шеннона (посимвольного)

Заметно, что на гистограмме нет четко выраженных пиков. В общем – гистограмма выглядит очень однородной, лишь с малыми пиками и просадками.

Теперь рассмотрим частоту встречаемости букв в тексте, зашифрованном при помощи шифра Шеннона (побитового).

Таблица 5 – частота встречаемости букв в тексте, зашифрованном при помощи шифра Шеннона (побитового)

Буква	А	Б	В	Г	Д	Е	Ж	З	И	Й	К
Частота встречаемости, раз	29	33	31	24	23	29	51	33	28	45	38
Относительная частота встречаемости, %	2.76	3.14	2.95	2.28	2.19	2.76	4.85	3.14	2.66	4.28	3.62
Буква	Л	М	Н	О	П	Р	С	Т	У	Ф	Х
Частота встречаемости, раз	38	36	33	17	45	40	30	34	19	38	44
Относительная частота встречаемости, %	3.62	3.43	3.14	1.62	4.28	3.81	2.85	3.24	1.81	3.62	4.19

Продолжение таблицы 5

Буква	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я	
Частота встречаемости, раз	23	30	30	37	34	31	34	37	30	27	
Относительная частота встречаемости, %	2.19	2.85	2.85	3.52	3.24	2.95	3.24	3.52	2.85	2.57	

Рисунок 16 – Гистограмма встречаемости букв в тексте, зашифрованном при помощи шифра Шеннона (побитового)

Заметно, что на гистограмме нет четко выраженных пиков. В общем – гистограмма выглядит достаточно однородной, лишь с малыми пиками и просадками.

Теперь рассмотрим частоту встречаемости букв в тексте, зашифрованном при помощи ГОСТ 28147-89 (Режим Гаммирования).

Таблица 6 – частота встречаемости букв в тексте, зашифрованном при помощи ГОСТ 28147-89 (Режим Гаммирования)

Буква	А	Б	В	Г	Д	Е	Ж	З	И	Й	К
Частота встречаемости, раз	34	24	44	41	33	25	29	33	28	38	30
Относительная частота встречаемости, %	3.24	2.28	4.19	3.9	3.14	2.38	2.76	3.14	2.66	3.62	2.85
Буква	Л	М	Н	О	П	Р	С	Т	У	Ф	Х
Частота встречаемости, раз	35	28	45	31	43	29	33	27	28	25	27
Относительная частота встречаемости, %	3.33	2.66	4.28	2.95	4.09	2.76	3.14	2.57	2.66	2.38	2.57
Буква	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я	
Частота встречаемости, раз	40	33	27	31	31	33	42	31	38	35	
Относительная частота встречаемости, %	3.81	3.14	2.57	2.95	2.95	3.14	4	2.95	3.62	3.33	

Рисунок 16 – Гистограмма встречаемости букв в тексте, зашифрованном при помощи шифра Шеннона (побитового)

Заметно, что на гистограмме нет четко выраженных пиков. В общем – гистограмма выглядит очень однородной, лишь с редкими и малыми пиками или просадками.

2.2.2 Построение и анализ гистограмм относительных частот встречаемости биграмм в исходном тексте и зашифрованных текстах

Разобьем наш открытый текст на биграммы, отсортируем и посмотрим, какие самые частые сочетания в нём встречаются.

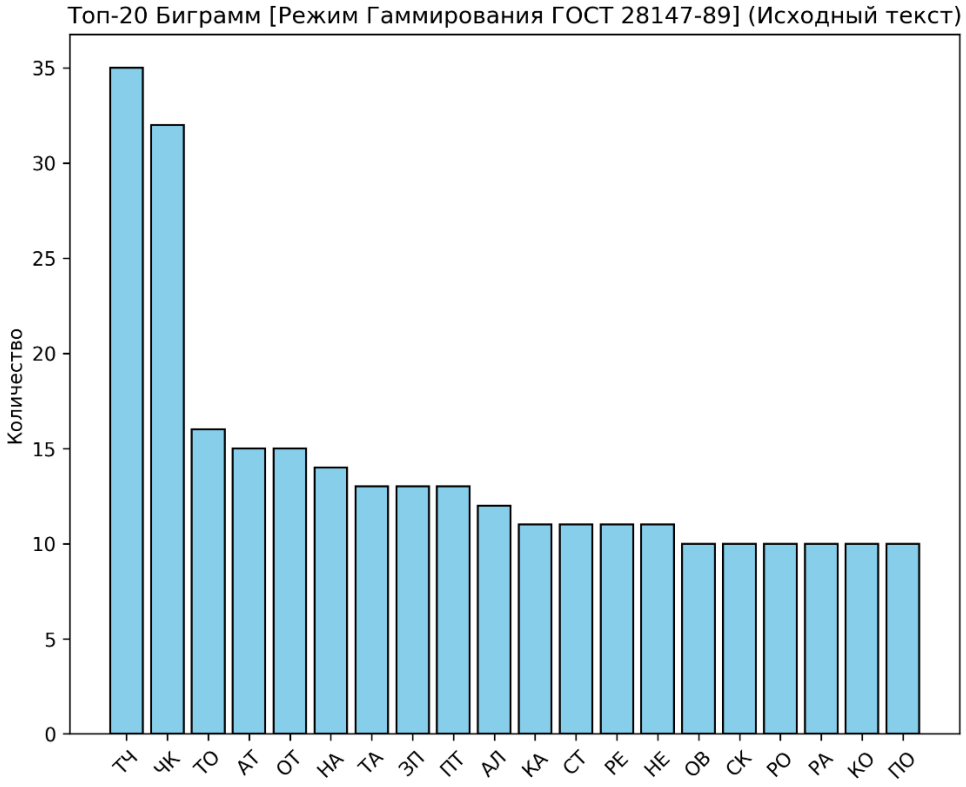


Рисунок 17 – Гистограмма встречаемости биграмм в исходном тексте

Мы видим, что гистограмма получается достаточно неровной и с четко выраженными четырьмя пиками – это биграммы «ТЧ», «ПТ», «ЗП», «ЧК». Если их объединять, то вполне понятно, что эти биграммы соответствуют буквенному обозначению знаков препинания «ЗПТ» и «ТЧК».

Рассмотрим, какие биграммы чаще всего встречаются в тексте, зашифрованном при помощи выбранных шифров. Для удобства будем располагать гистограмму оригинала рядом с гистограммой биграмм каждого зашифрованного текста.

Начнем с шифра Вернама (посимвольного):

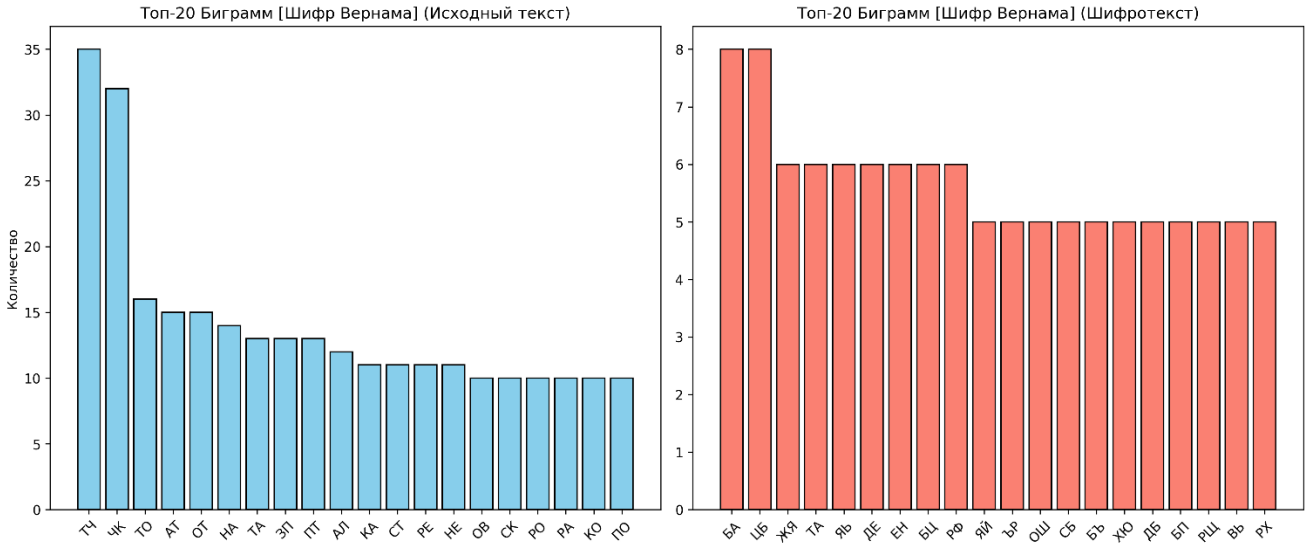


Рисунок 18 – Гистограмма встречаемости биграмм в шифре Вернама (посимвольном) и в исходном тексте

Мы видим, что гистограмма получается достаточно ровной. Среди самых частых биграмм нет особо выделяющихся биграмм, однако их количество (6-8 штук) – это уже немало, что указывает на меньшую криптографическую стойкость шифра Вернама, что было ранее отмечено в главе 1 курсовой работы.

Рассмотрим побитовый вариант шифра Вернама:

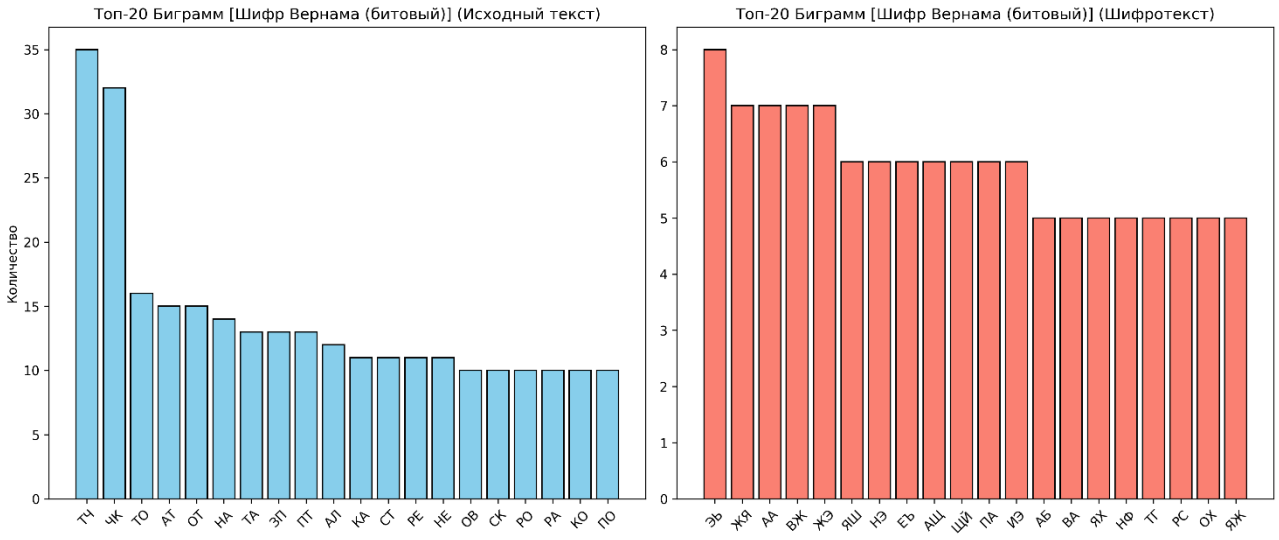


Рисунок 19 – Гистограмма встречаемости биграмм в шифре Вернама (побитовом) и в исходном тексте

Мы видим, что гистограмма получается достаточно ровной. Среди самых частых биграмм нет особо выделяющихся биграмм, однако их количество (6-8 штук) – снова указывает на меньшую криптографическую стойкость шифра Вернама.

Рассмотрим шифр Шеннона (посимвольный):

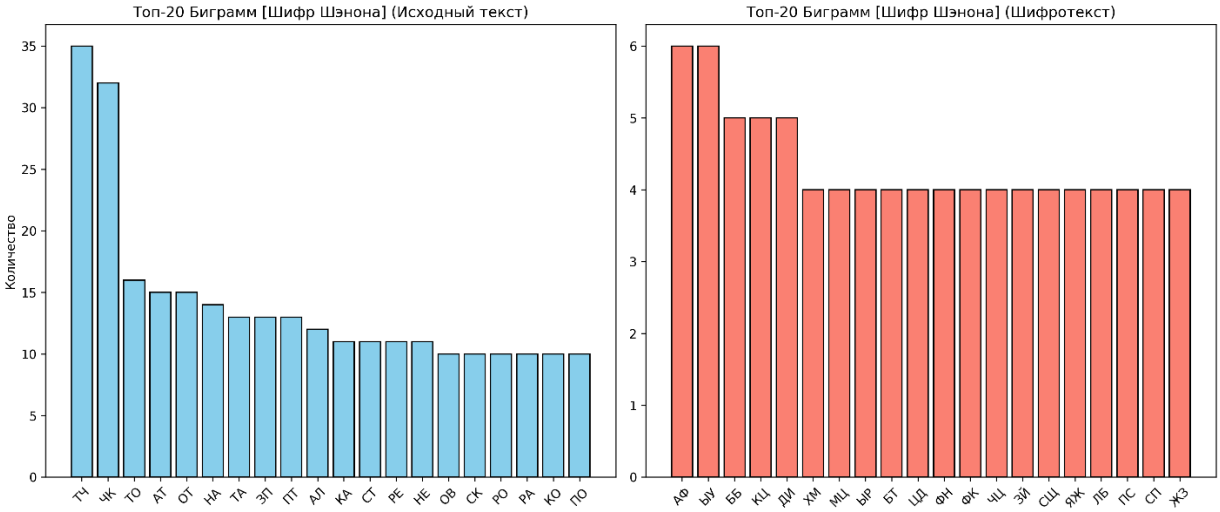


Рисунок 20 – Гистограмма встречаемости биграмм в шифре Шеннона (посимвольном) и в исходном тексте

Мы видим, что гистограмма получается достаточно ровной. Среди самых частых биграмм нет особо выделяющихся биграмм, а их количество (6 повторений максимум) – указывает на большую криптографическую стойкость, чем у шифра Вернама.

Рассмотрим побитовый вариант шифра Шеннона:

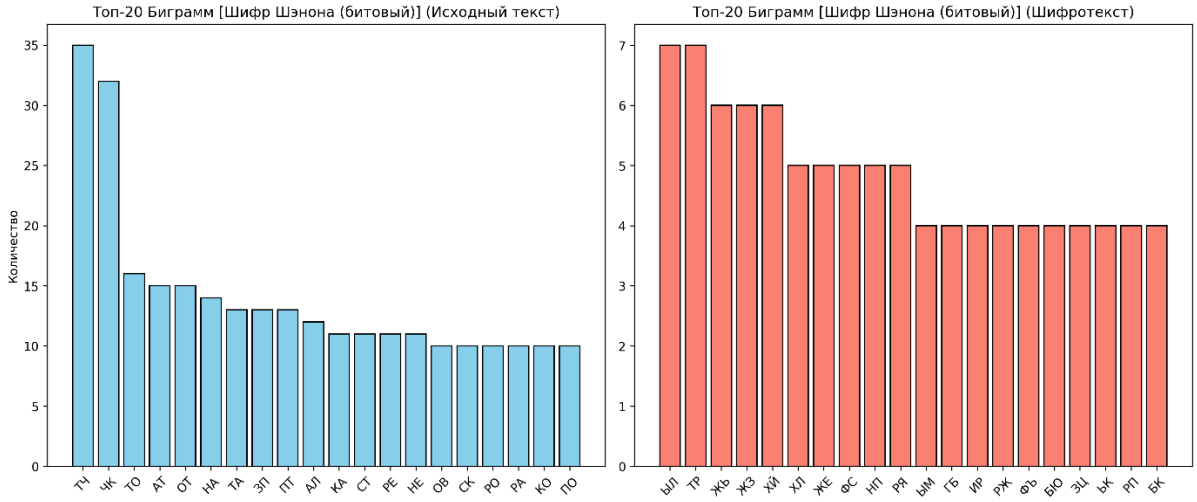


Рисунок 21 – Гистограмма встречаемости биграмм в шифре Шеннона (побитовом) и в исходном тексте

Мы видим, что гистограмма получилась менее ровной, чем в посимвольном варианте. Среди самых частых биграмм нет особо выделяющихся биграмм, но их количество (7 повторений максимум) – указывает, что в данном примере при заданных параметрах линейного конгруэнтного генератора и заданном исходном тексте получаемый шифртекст имеет большее совпадение биграмм, чем посимвольный вариант.

Рассмотрим ГОСТ 28147-89 (Режим Гаммирования):

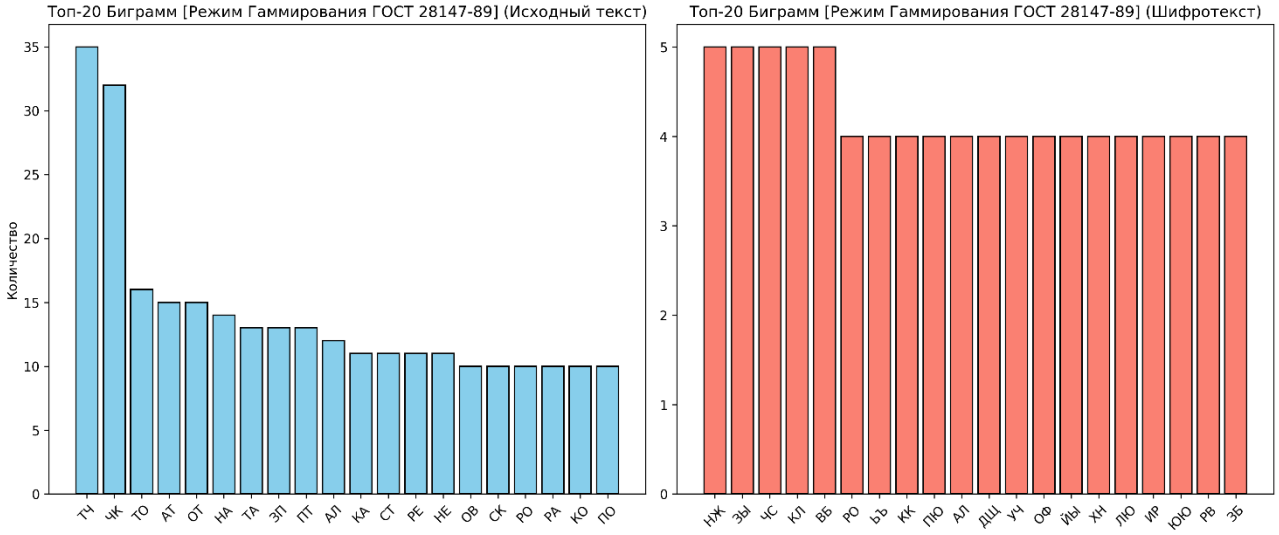


Рисунок 22 – Гистограмма встречаемости биграмм в ГОСТ 28147-89 (Режим Гаммирования) и в исходном тексте

Посмотрев на гистограмму, мы сразу можем заметить, что гистограмма получается более ровной, чем все предыдущие. Среди самых частых биграмм нет выделяющихся биграмм, а их количество (5 повторений максимум) – указывает на большую криптографическую стойкость данного шифра.

2.2.3 Построение и анализ гистограмм относительных частот встречаемости триграмм в исходном тексте и зашифрованных текстах

Аналогично предыдущему пункту, разобьем наш открытый текст на триграммы, отсортируем и посмотрим, какие самые частые сочетания в нём встречаются.

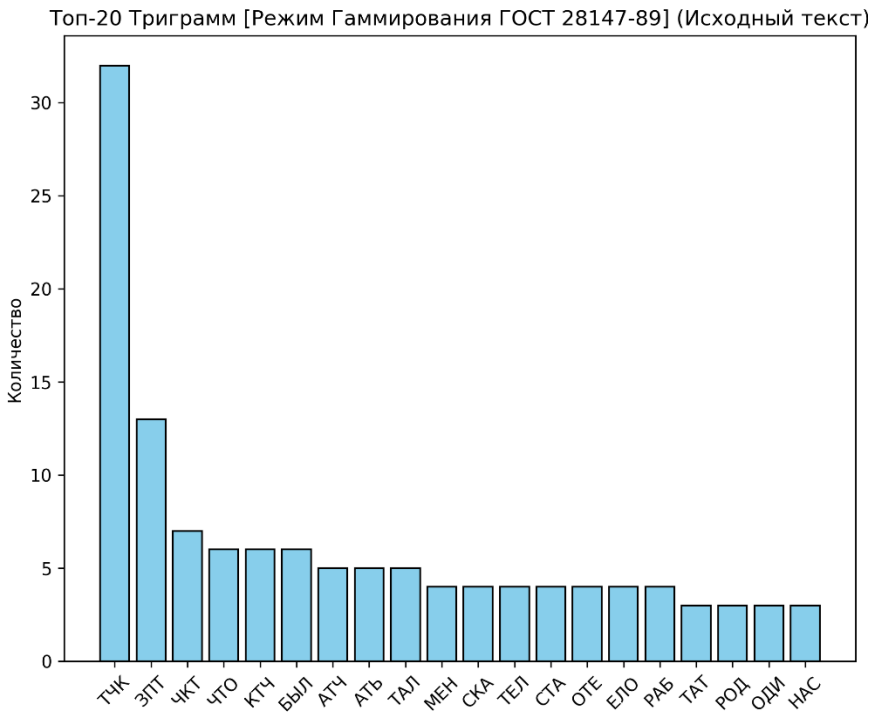


Рисунок 23 – Гистограмма встречаемости триграмм в исходном тексте

Мы видим, что гистограмма получается очень неровной, с четко выраженными пиками – это триграммы «ТЧК», «ЗПТ», «ЧТО», «БЫЛ», «РАБ», «НАС». Если их объединять, то вполне понятно, что эти биграммы соответствуют буквенному обозначению знаков препинания «ЗПТ» и «ТЧК», а по триграмме «РАБ» – что в тексте, вероятно, много раз упоминалось слово «работа» и его производные – что соответствует выбранному тексту.

Рассмотрим, какие триграммы чаще всего встречаются в тексте, зашифрованном при помощи выбранных шифров. Для удобства будем располагать гистограмму оригинала рядом с гистограммой триграмм каждого зашифрованного текста.

Начнем с шифра Вернама (посимвольного):

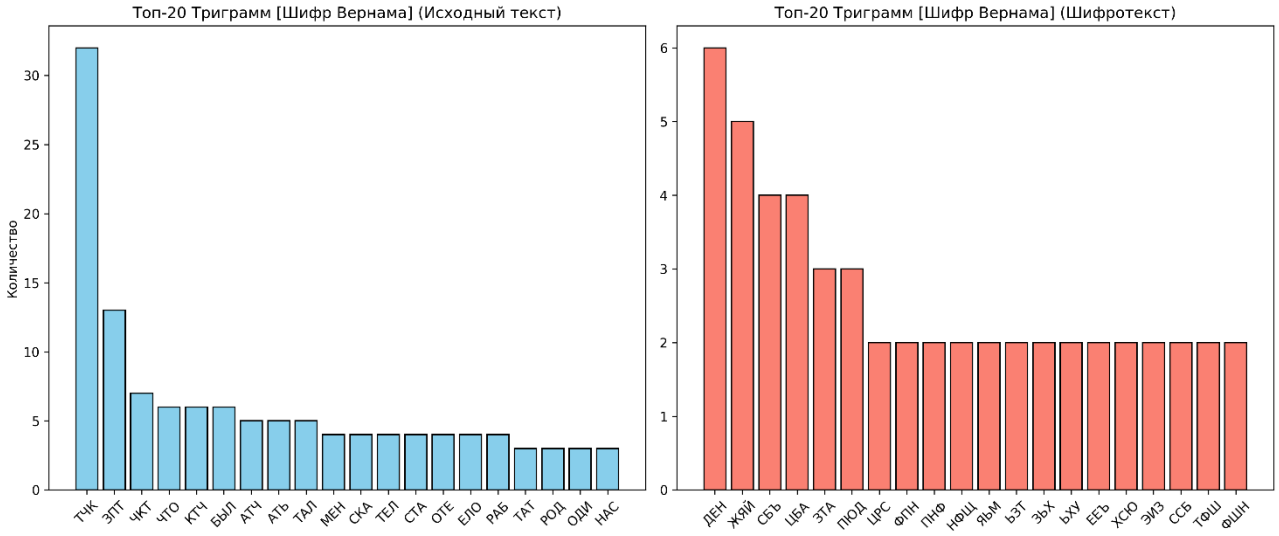


Рисунок 24 – Гистограмма встречаемости триграмм в шифре Вернама (посимвольном) и в исходном тексте

Мы видим, что гистограмма получается более сглаженной, чем оригинал, однако присутствуют свои пики. Среди самых частых триграмм хоть и имеются выделяющиеся трииграммы в заметном количестве (3-6 штук) – напрямую содержание текста они не выдают. Тем не менее, 3-6 штук – это уже немало, что указывает на меньшую криптографическую стойкость шифра Вернама, что было ранее отмечено в главе 1 курсовой работы.

Рассмотрим побитовый вариант шифра Вернама:

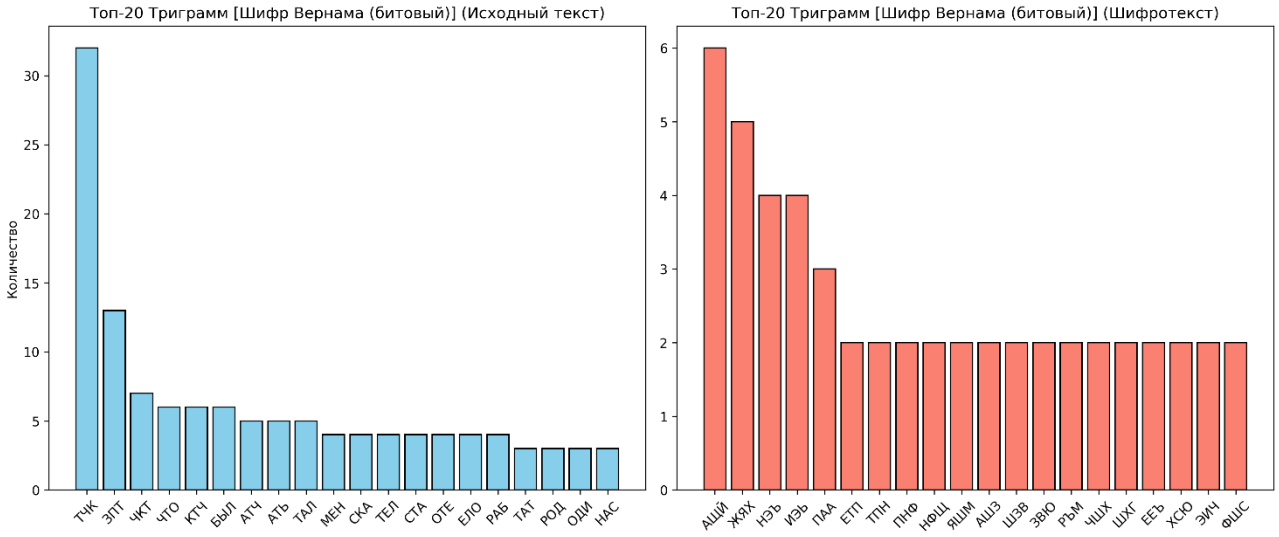


Рисунок 25 – Гистограмма встречаемости триграмм в шифре Вернама (побитовом) и в исходном тексте

Аналогично посимвольному варианту, мы видим, что гистограмма получается более сглаженной, чем оригинал, со своими пики. Тем не менее сохраняется та же проблема с тем, что повторы триграмм встречаются в немалом количестве (4-6 штук).

Рассмотрим шифр Шеннона (посимвольный):

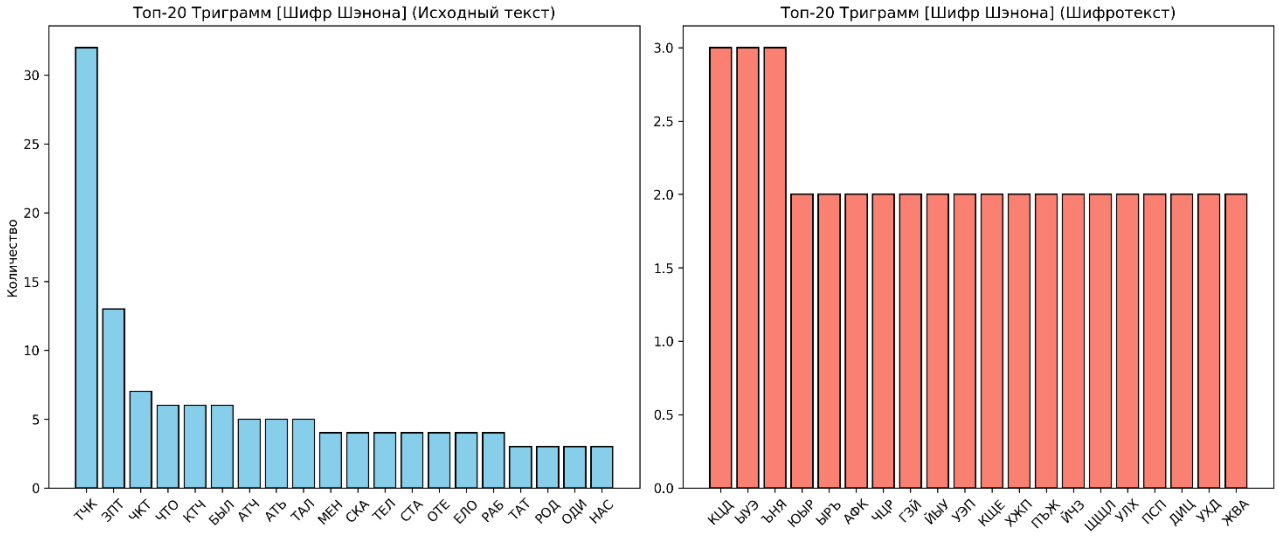


Рисунок 26 – Гистограмма встречаемости триграмм в шифре Шеннона (посимвольном) и в исходном тексте

Посмотрев на гистограмму, мы сразу же можем заметить, что гистограмма получается сильно сглаженной. Среди самых частых триграмм хоть и имеются выделяющиеся триграммы, однако их количество несущественно отличается от оставшихся триграмм (3 штуки, а не 2). Это указывает на большую криптографическую стойкость шифра Шеннона, а также на его превосходство над шифром Вернама, что было ранее отмечено в главе 1 курсовой работы.

Рассмотрим побитовый вариант шифра Шеннона:

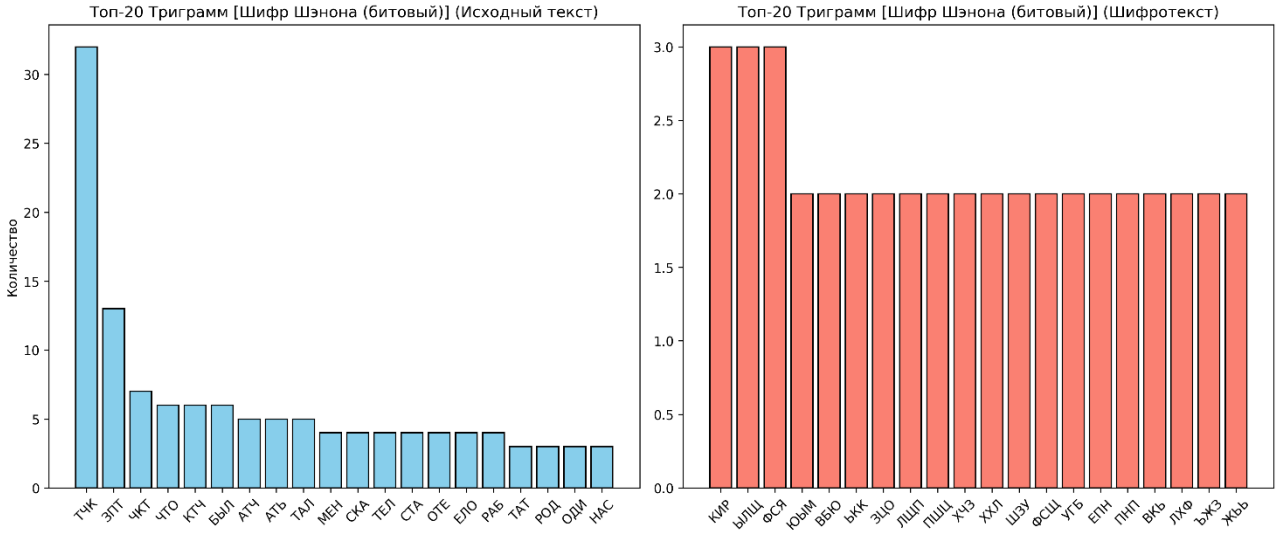


Рисунок 27 – Гистограмма встречаемости триграмм в шифре Шеннона (побитовом) и в исходном тексте

Аналогично гистограмме триграмм посимвольного шифра Шеннона – особых пиков не наблюдается. Общее количество триграмм также равномерно распределено, о чем говорит максимальное число повторений триграмм – 3 штуки.

Рассмотрим ГОСТ 28147-89 (Режим Гаммирования):

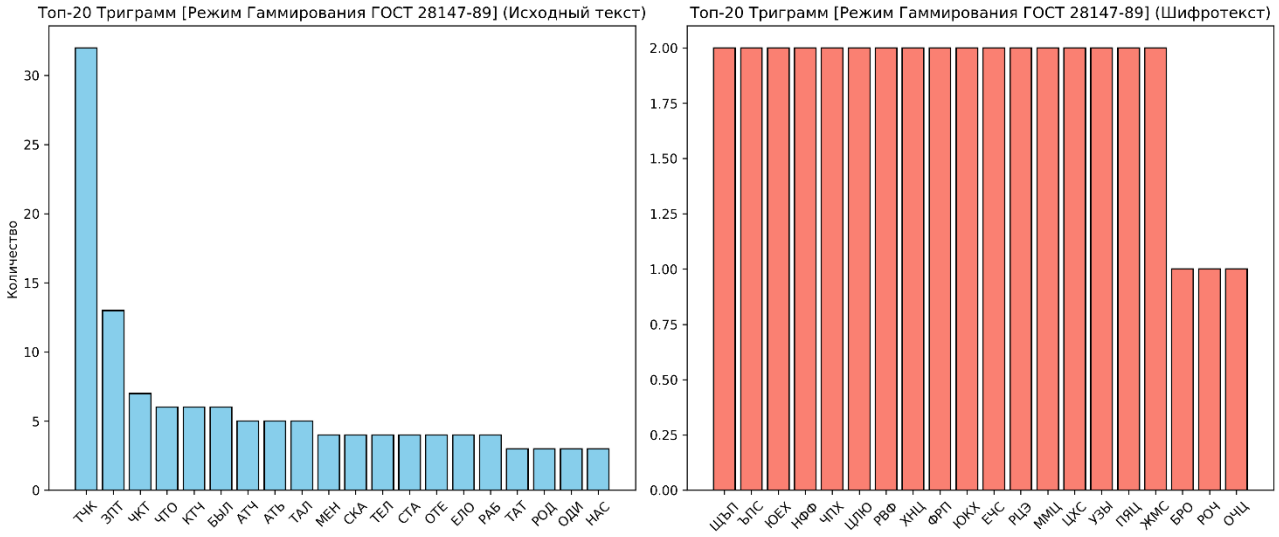


Рисунок 28 – Гистограмма встречаемости триграмм в ГОСТ 28147-89 (Режим Гаммирования) и в исходном тексте

По гистограмме сразу заметно, что среди всех шифров гаммирования, наиболее равномерно распределены триграммы у ГОСТ 28147-89 (Режим Гаммирования). Максимально

число повторений триграмм – 2 раза, так еще и таких триграмм – всего 17 штук на весь шифртекст.

2.3 Криптоанализ исследуемых шифров

Как уже отмечалось выше, криптоанализ – наука о методах и способов вскрытия шифров. В этом разделе мы рассмотрим, какие методы теоретически могут помочь вскрыть рассматриваемые в данной курсовой работе шифры.

2.3.1 Криптоанализ шифра Вернама и Шеннона

Если соблюдать все условия, указанные в главе 1, касающиеся криптографической стойкости шифра Вернама и шифра Шеннон, которые перечислил Клод Шеннон, то эти шифры становятся практически невзламываемыми. Однако на то они и условия, что их несоблюдение ведет к появлению уязвимостей.

Если для шифрования двух текстов была применена одна и та же гамма (одноразовый шифр-блокнот, например, был использован дважды), то при использовании битовой гаммы можно получить сумму (логическое ИСКЛЮЧАЮЩЕЕ ИЛИ – XOR) от двух исходный текстов (так как гамма наложится на гамму и даст ноль). Далее можно пытаться подбирать различные слова, которые накладываясь «друг на друга» дают такой же результат. Если же одно из сообщений уже хотя бы частично известно – то второе расшифровать – не составляет труда.

В случае ключа конечной длины, период которого повторяется N-ое число раз на всём тексте, можно применять частотный анализ по соответствующим битам или символам.

Если же стали известны некоторые параметры применяемого линейного конгруэнтного генератора – можно перебрать множество вариантов генерируемых последовательностей, пытаясь затем расшифровать начало сообщения, а в случае успеха – продолжить генерировать последовательность (гамму), пока она вся не будет «снята» с шифртекста.

2.3.2 Криптоанализ ГОСТ 28147-89 (Режим Гаммирования)

Аналогично прошлому пункту, если для шифрования двух текстов была применена одна и та же гамма (один и тот же ключ с синхропосылкой были использованы дважды), то можно получить сумму двух исходный текстов. Дальнейший криптоанализ должен строится на том, что одно из сообщений хотя бы частично известно.

Тем не менее, при соблюдении требований разумного использования шифра (не использовать одни и те же ключи и синхропосылки) ГОСТ 28147-89 и по сей день является достаточно криптографически стойким шифром.

2.4 Рекомендации по использованию исследованных шифров для защиты информации

Таким образом, на основании изложенного выше материала можем сказать о том, что для шифров, рассмотренных в данной курсовой работе, можно привести следующие рекомендации по использованию:

1. Шифр Вернама – требует использования достаточно большого ключа в виде текста, что не удобно в плане использования, а потому его применение не имеет смысла, за исключением обучения криптоанализу и простого ознакомления, как этапа развития криптографии и шифров гаммирования.
2. Шифр Шеннона – требует правильного подбора параметров линейного конгруэнтного генератора для генерации гаммы гарантированно большей длины, чем сам текст. Опять же, это не очень удобно, так как бывают такие параметры, которые могут давать периоды и более 1000 бит, однако изменение одного из параметров может привести к тому, что период сократится кратно. Трудности в подборе параметров линейного конгруэнтного генератора и делают шифр Шеннона практически – не удобным шифром, хоть его криптостойкость и была доказана математически.
3. ГОСТ 28147-89 (Режим Гаммирования) – хороший шифр гаммирования при соблюдении простого правила: «один текст – один ключ». Хоть он и считается формально устаревшим, ведь на замену ему пришли «МАГМА» и «КУЗНЕЧИК», но он все ещё используется в некоторых системах (в силу необходимости поддержки legacy-систем), обеспечивая их криптографическую защиту.

Вывод по главе 2

В этой главе было произведено сравнение криптостойкости алгоритмов шифрования шифра Вернама, шифра Шеннона и ГОСТ 28147-89 (Режим Гаммирования), а также выработаны рекомендации по их применению для защиты информации.

Большую часть главы занял гистограммный анализ данных шифров. В ходе него было выявлено, какой из шифров генерирует менее криптостойкий шифртекст, а какой – более криптостойкий. По итогам анализа стало очевидным, что шифр Вернама в любом сравнении с шифром Шеннона – слабее криптографически. ГОСТ 28147-89 (Режим Гаммирования) в свою очередь показывает, что он явно был создан позже остальных рассматриваемых шифров. Реализуя саму по себе более сложную схему, он, как итог, генерирует более «случайный» шифртекст, который практически не имеет уязвимостей.

Результаты этой главы показывают, что шифр Вернама, например, имеет больше историческую ценность, чем практическую, а шифр Шеннона с достаточной криптостойкостью – не так просто реализовать из-за особенности линейных конгруэнтных генераторов.

Наилучшую криптостойкость показал ГОСТ 28147-89 (Режим Гаммирования), который благодаря большому количеству раундов со сложной математической моделью имеет высокую криптостойкость, которая и позволила на его основе создать, например, шифр «МАГМА», который актуален в наше время в РФ.

ЗАКЛЮЧЕНИЕ

В курсовой работе были рассмотрены шифры гаммирования: шифр Вернама, шифр Шеннона и ГОСТ 28147-89 (Режим Гаммирования). Были изучены их алгоритмы, проведено шифрование большого (1000+ символов) текста, выполнен гистограммный анализ встречаемости символов текста, биграмм и триграмм, а также сформулированы рекомендации по использованию рассмотренных шифров.

Было выявлено, что основная угроза для шифров гаммирования состоит в повторном использовании гаммы или ключа. Если одна и та же гамма применяется для шифрования двух различных открытых текстов, то злоумышленник, вычислив логическое ИСКЛЮЧАЮЩЕЕ ИЛИ (XOR) двух шифртекстов, получает XOR двух исходных текстов, что позволяет провести криптоанализ. Уязвимость также может возникнуть, если длина ключа конечна и он повторяется по всему тексту, что позволяет применять частотный анализ для вскрытия шифртекста.

Шифр Вернама продемонстрировал худшие показатели, в сравнении остальными шифрами и, в частности, шифром Шеннона. Он имеет скорее историческую ценность, чем практическую. Шифр Шеннона показал хорошую криптостойкость, но его криптостойкость шифра напрямую зависит от правильного подбора параметров линейного конгруэнтного генератора. ГОСТ 28147-89 (Режим Гаммирования) показал наилучшую криптостойкость. Благодаря более сложной схеме (использование сети Фейстеля, S-блоков и 32 раундов в генераторе гаммы), он генерирует наиболее «случайный» шифртекст, с наиболее равномерным распределением символов, биграмм и триграмм. Он, несмотря на формальное устаревание, всё ещё достаточно криптографически стоек и послужил основой для разработки более современного российского стандарта — шифра «МАГМА».

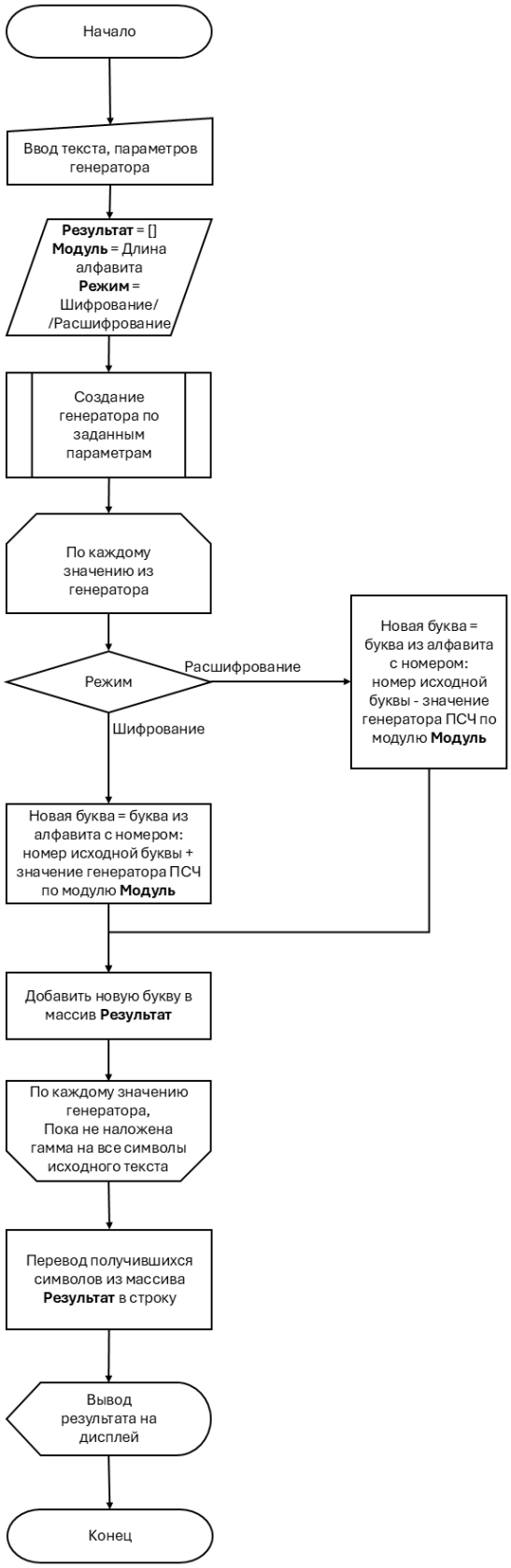
В итоге можно сделать вывод, что на сегодняшний день наиболее применимым среди рассмотренных шифров гаммирования является ГОСТ 28147-89, при условии соблюдения простого правила: "один текст – один ключ". Шифры Вернама и Шеннона имеют скорее историческое или учебное значение, а их практическое применение ограничено требованиями к генерации случайной или псевдослучайной гаммы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

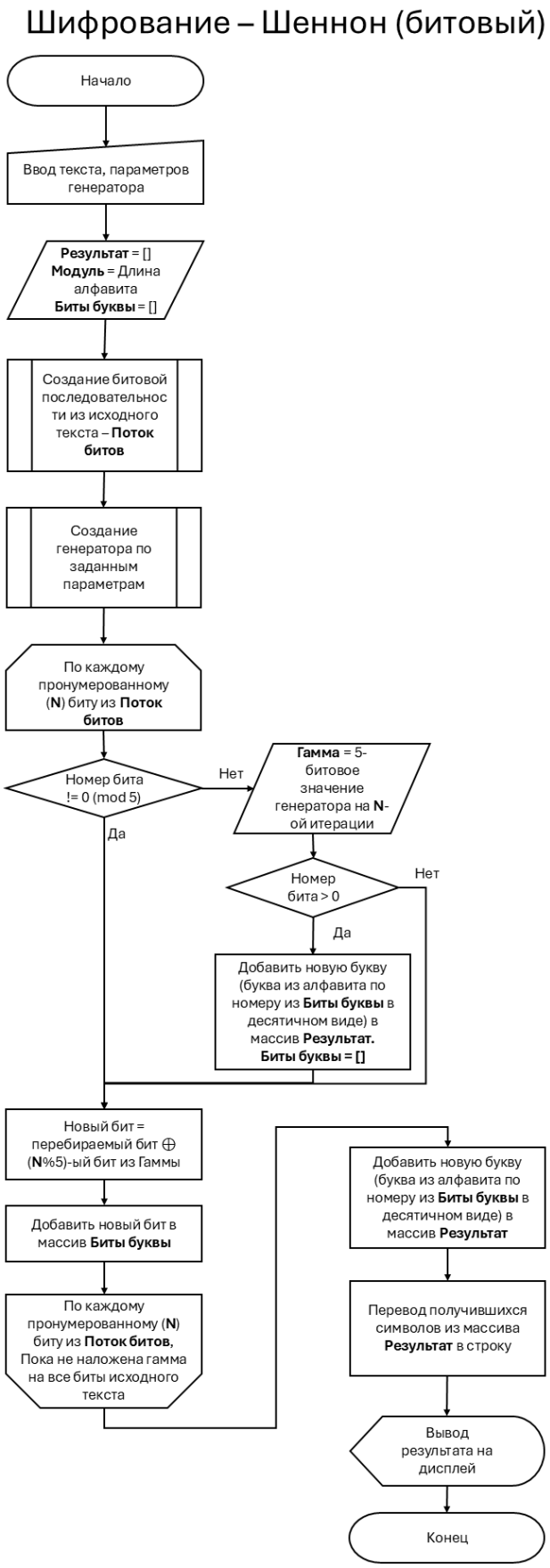
1. Хахина, А.М. Шифры: прошлое и современность / А. М. Хахина, В. Д. Покровский, А. В. Краев // Заметки ученого. — 2020. — № 2. — С. 169-171. — ISSN 2713-0142. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/journal/issue/321518 (дата обращения: 12.12.2025). — Режим доступа: для авториз. пользователей.
2. Иванов, Ю.С. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ МЕТОДА АССИМЕТРИЧНОГО ШИФРОВАНИЯ / Ю.С. Иванов, Т.И. Кирюшко // Вестник научного общества студентов, аспирантов и молодых ученых. — 2014. — № 1. — С. 145-154. — ISSN 9999-7444. — Текст : электронный // Лань : электронно-библиотечная система.— URL: https://e.lanbook.com/journal/issue/294138 (дата обращения: 14.12.2022). — Режим доступа: для авториз. пользователей.
3. Бутакова Н.Г., Федоров Н.В. Криптографические методы и средства защиты информации: учеб. пособие / Н.Г.Бутакова, Н.В.Федоров. − СПб.: ИЦ «Интермедия», 2019. – 384 с.
4. Шеннон, Клод [Электронный ресурс] // Википедия : свободная энциклопедия. — Электрон. дан. — Режим доступа: https://ru.wikipedia.org/wiki/Шеннон,_Клод (дата обращения: 12.12.2025).

ПРИЛОЖЕНИЕ А

Шифрование – Шеннон (символьный)

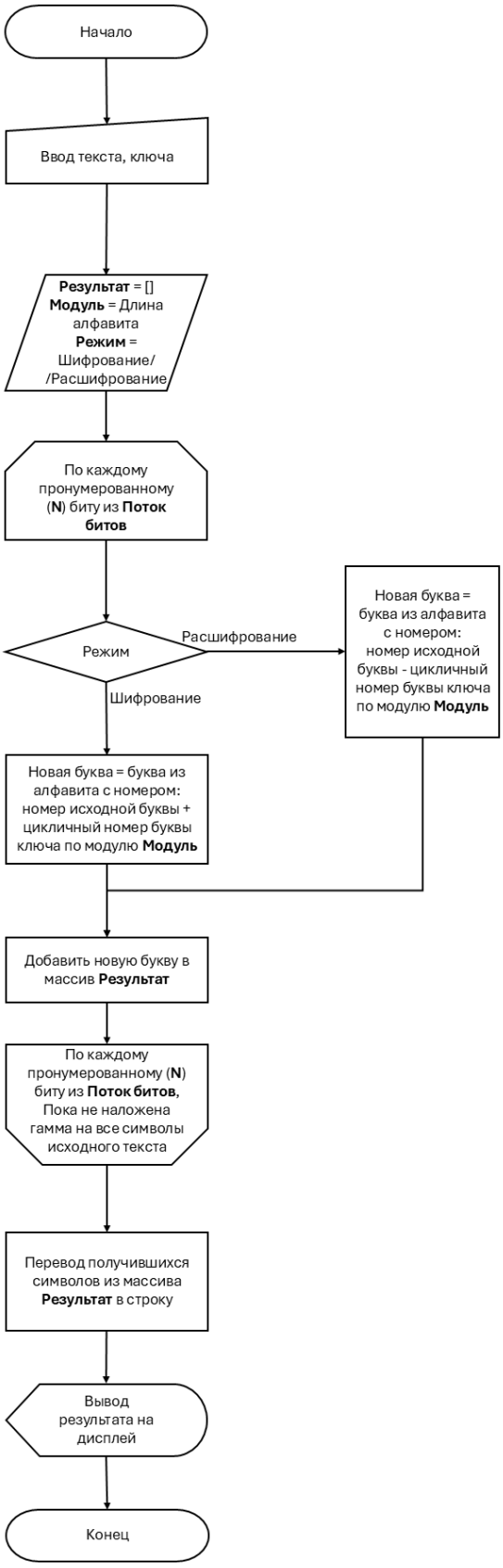


ПРИЛОЖЕНИЕ Б



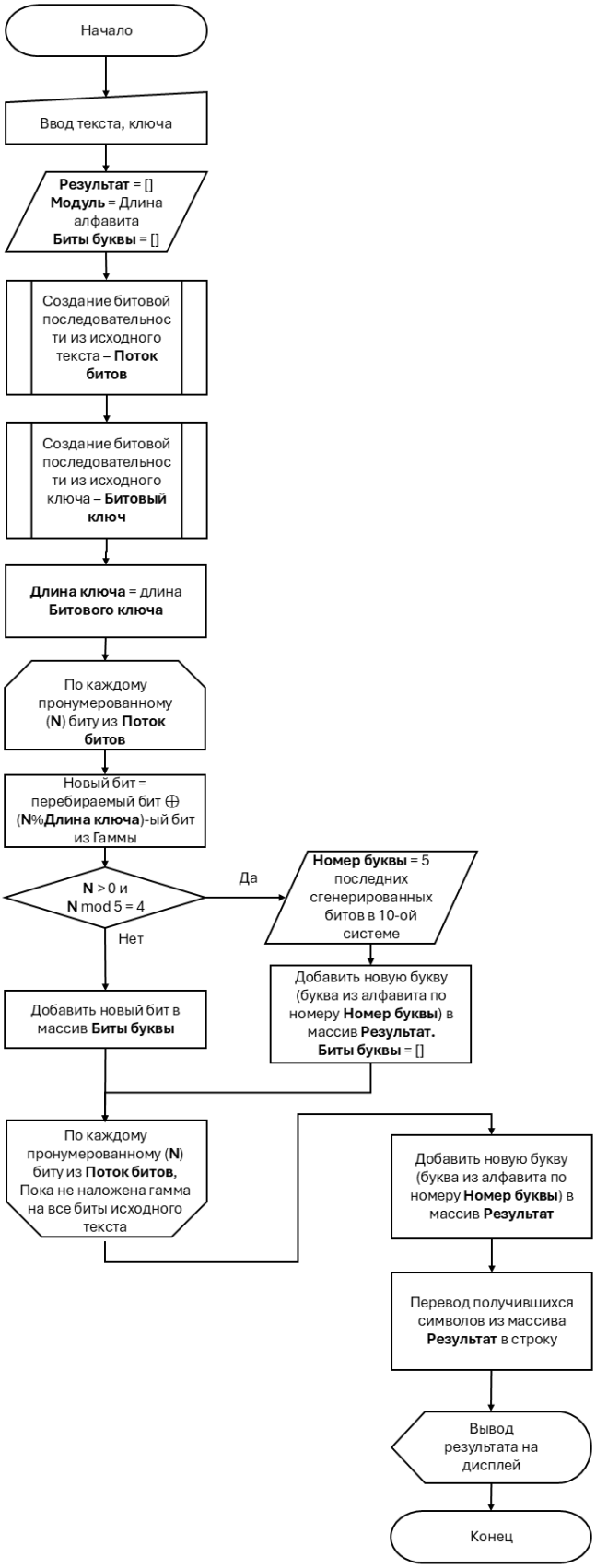
ПРИЛОЖЕНИЕ В

Шифрование – Вернам (символьный)



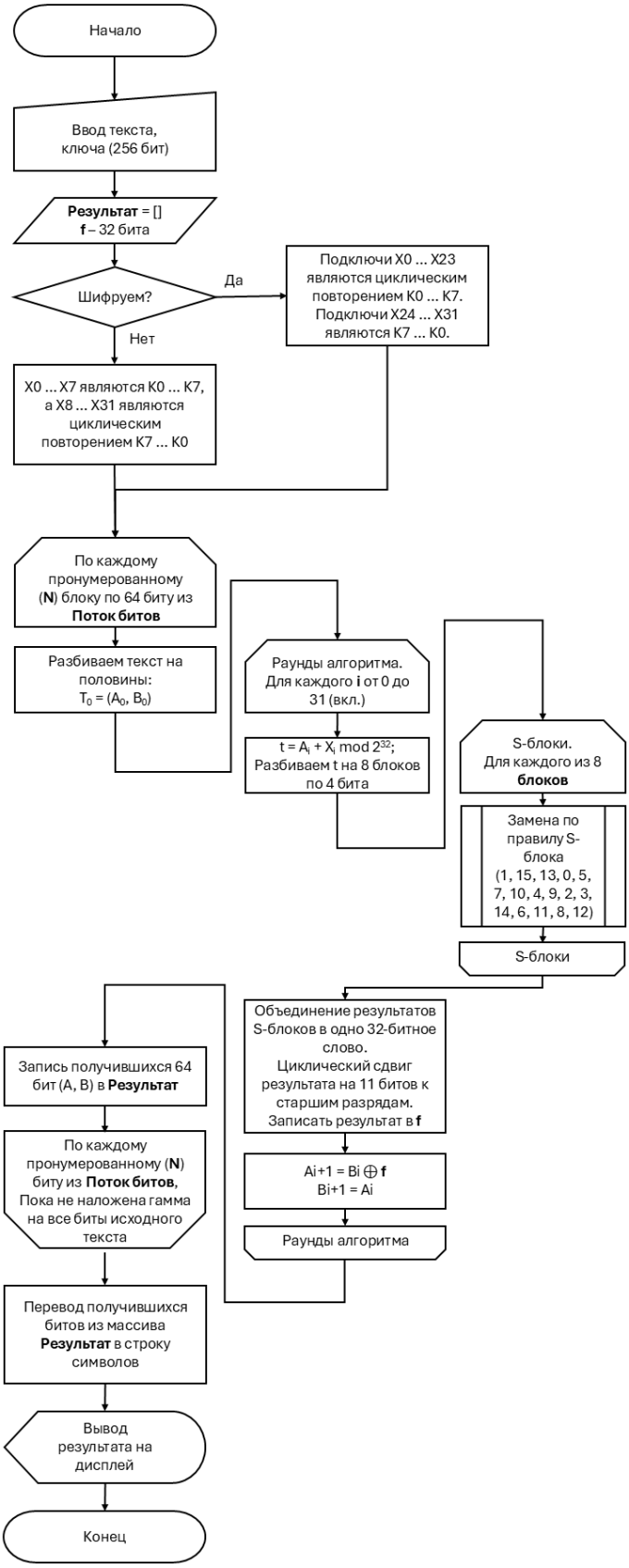
ПРИЛОЖЕНИЕ Г

Шифрование – Вернам (битовый)



ПРИЛОЖЕНИЕ Д

ГОСТ 28147-89 (Простая замена)



ПРИЛОЖЕНИЕ Е

ГОСТ 28147-89 (Гаммирование)

ПРИЛОЖЕНИЕ Ж

```
# Бройдо Глеб #
#    ИБ-31    #
#     2025    #
###############

import os, platform, random, math, random, textwrap
from typing import List, Callable

WIDTH = 30
PLACEHOLDER = "..."
LONG_TEXT = """
Меня зовут Вито Скалетта. Я родился на Сицилии в двадцать пятом году. Этот малыш я.
Стою с родителями и сестрой Франческой у нашего старого дома. Я мало что помню о
тех временах... Разве что жизнь была тяжелой. А потом отец решил, что нам пора
уезжать. Покинуть Сицилию. Пересечь океан и начать новую жизнь в Америке.

Никогда в жизни я не видел ничего, подобного Эмпайр Бэй. Он был прекрасен. Впрочем,
трудно представить нечто более скверное и отвратительное, чем наша новая конура.
Американская мечта. Скорее уж кошмар. Отец стал работать в порту у человека,
который перевез нас сюда. Он работал как проклятый, зарабатывал гроши и почти все
спускал в кабаке. Со временем меня отдали в школу. Мне нужен был английский, но
черта с два я бы заговорил на нем с итальянцами. Там я и встретил Джо...

Постепенно мы стали лучшими друзьями. У обоих свистело в кармане, а пристойной
работы не было и мы затеяли свое скромное дело...

Что ж, тогда нам не повезло. На дворе стоял сорок третий год. Шла война, и для
высадки на Сицилии нужны были солдаты, говорящие по итальянски. Мне было
восемнадцать, и в тюрьму я не хотел. Кто сказал, что нельзя вернуться на родину.
""".replace('\n', '')
OK_SYMBOL = '✅'
NOT_OK_SYMBOL = '❌'

##############################################################################
# text_utils ---------------------------------------------------------------#
ALPHABET = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"                                #
VALID_SYMBOLS = set(ALPHABET + ",. ")                                        #
PROVERB  = "Леопард не может изменить своих пятен."                          #
#---------------Функции для "стандартизации" строки под шифрование---------------#
def remove_special_chars(text: str) -> str:                                  #
    '''                                                                      #
    Убирает пробелы. Заменяет точки и запятые на ТЧК и ЗПТ.                  #
                                                                             #
    Также делает все буквы заглавными.                                       #
    '''                                                                      #
    new_text = []                                                            #
    text = text.upper()                                                      #
```

```
    for i in text:                                                          #
        if i not in VALID_SYMBOLS:                                          #
            pass                                                            #
        new_text.append(                                                    #
            i.replace(',', 'ЗПТ').replace('.', 'ТЧК').replace(' ', '')      #
        )                                                                   #
    return ''.join(new_text)                                                #
                                                                            #
def insert_spaces(text: str) -> str:                                        #
    '''                                                                     #
    Форматирует строку в вид:                                               #
                                                                            #
    5 символов - пробел - 5 символов - пробел ...                           #
    '''                                                                     #
    new_text = []                                                           #
    for i, v in enumerate(text):                                            #
        new_text.append(v)                                                  #
        if (i + 1) % 5 == 0:                                                #
            new_text.append(' ')                                            #
    return ''.join(new_text)                                                #
                                                                            #
def standartise(text: str) -> str:                                          #
    '''                                                                     #
    Форматирует строку в строку без пробелов и с заменой на ТЧК и ЗПТ.      #
                                                                            #
    Затем - приводит к виду: 5 символов - пробел - 5 символов - пробел ...  #
    '''                                                                     #
    return insert_spaces(remove_special_chars(text))                        #
#-----------------------------------------------------------------------------#
#############################################################################
# Функции для перевода в/из бинарной репрезентации

def symbol_to_bits(symbol: str) -> str:
    '''
    Возвращает номер символа в алфавите в двоичной репрезентации.
    '''
    return f"{ALPHABET.find(symbol):05b}"
def string_to_bits(text: str) -> str:
    '''
    Переводит посимвольно целую строку в ее битовую репрезентацию.
    '''
    new_text = []
    for i in text:
        new_text.append(symbol_to_bits(i))
    return ''.join(new_text)
def bits_to_symbol(bits: str) -> str:
    '''
    Возвращает саму букву из ее бинарной кодировки.
    '''
    # print(bits)
    return ALPHABET[int(bits, base=2)]
```

```python
def bits_to_string(bits: str, block_size: int=5) -> int:
    '''
    Переводит поблочно (5 битов) целую последовательность битов в строку символов.
    '''
    new_text = []
    for i in range(0, len(bits), block_size):
        new_text.append(bits_to_symbol(bits[i:i + block_size]))
    return ''.join(new_text)

##################################################
# Для генерации больших простых чисел

def isPrimeTrivial(num: int) -> bool:
    '''
    Trivial check whether given NUMBER is PRIME or NOT

    It checks all integers up to sqrt(num), including it whole part

    :param num: Number being checked
    :return: True (is indeed PRIME) or False (not a PRIME)
    '''

    if num < 2: return False

    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0:
            return False

    return True
def primeSieve(size: int) -> List[int]:
    '''
    Makes a Sieve of Eratosthenes using SIZE numbers (0, 1, ..., SIZE)

    :param size: Largest integer being checked
    :return: List of prime numbers <= Size
    '''
    size += 1

    sieve: List[bool] = [True] * size
    sieve[0], sieve[1] = False, False # 0 and 1 are not PRIMES

    for i in range(2, int(math.sqrt(size)) + 1):
        pointer = i * 2
        while pointer < size:
            sieve[pointer] = False
            pointer += i

    primes: List[int] = []

    for i in range(size):
        if sieve[i] == True:
```

```
            primes.append(i)

    return primes
def RabinMillerTest(num: int) -> bool:
    '''
    Checks whether given NUMBER is PRIME or NOT using Rabin-Miller Algorithm

    :param num: Number being checked
    :return: True (is indeed PRIME) or False (not a PRIME)
    '''

    if num % 2 == 0 or num < 2: return False
    if num == 3:                return True

    temp = num - 1
    times = 0

    while temp % 2 == 0:
        temp = temp // 2
        times += 1

    for _ in range(5): # Checking 5 times
        RNGint = random.randrange(2, num - 1)
        v = pow(RNGint, temp, num)
        if v != 1:
            i = 0
            while v != (num - 1):
                if i == (times - 1):
                    return False
                else:
                    i = i + 1
                    v = pow(v, 2, num)

    return True
LOW_PRIMES = primeSieve(1000)
def isPrime(num: int) -> bool:
    '''
    :param num: Number being checked
    :return: True (is indeed PRIME) or False (not a PRIME)
    '''
    if num < 2: return False

    for prime in LOW_PRIMES:
        if num == prime:
            return True
        if num % prime == 0:
            return False

    return RabinMillerTest(num)
def generateLargePrime(keysize: int = 1024) -> int:
    '''
```

```
    Generates a random (keysize) bit PRIME integer

    :param keysize: Amount of bits given to generating integer
    :return: Large Prime Integer (keysize) bits size
    '''
    while True:
        num = random.randrange( pow(2, keysize - 1), pow(2, keysize) )
        if isPrime(num):
            return num

#####################################################
# Имплементация функций по ГОСТ 28147-89

def gost_28147_89_simple_sub(text: str, key: int, encrypt: bool = True) -> str:
    """
    Реализует режим простой замены по ГОСТ 28147-89.
    Обеспечивает обратимость:
gost_28147_89_simple_sub(gost_28147_89_simple_sub(text, key), key, encrypt=False)
== биты текста.
    """

    # --- 1. Вспомогательная логика: Проверка и подготовка входных данных ---

    # Проверяем, передан ли уже поток битов ('0' и '1') или текст
    is_bit_stream = all(c in '01' for c in text) if text else False

    if is_bit_stream:
        bits = text
    else:
        bits = string_to_bits(remove_special_chars(text))

    # ГОСТ работает с блоками по 64 бита.
    # Если длина не кратна 64, дополняем нулями справа (padding).
    original_length = len(bits)
    pad_len = (64 - (len(bits) % 64)) % 64
    bits_padded = bits + '0' * pad_len

    # --- 2. Подготовка ключей ---

    # Разбиваем 256-битный ключ на 8 подключей по 32 бита (K0...K7)
    # Используем Little Endian (младшие биты - K0)
    subkeys = [(key >> (32 * i)) & 0xFFFFFFFF for i in range(8)]

    # Определение порядка ключей для 32 раундов
    if encrypt:
        # Шифрование:
        # Раунды 1-24: K0...K7 (повторить 3 раза)
        # Раунды 25-32: K7...K0 (в обратном порядке 1 раз)
        key_indices = list(range(8)) * 3 + list(range(7, -1, -1))
    else:
        # Расшифрование:
```

```
        # Для обратимости ГОСТа порядок ключей инвертируется специфическим образом:
        # Раунды 1-8: K0...K7 (прямой порядок 1 раз)
        # Раунды 9-32: K7...K0 (обратный порядок 3 раза)
        key_indices = list(range(8)) + list(range(7, -1, -1)) * 3

    # Таблица замен (S-box)
    S_BOX = (
        (12, 4, 6, 2, 10, 5, 11, 9, 14, 8, 13, 7, 0, 3, 15, 1),      # S1
        (6, 8, 2, 3, 9, 10, 5, 12, 1, 14, 4, 7, 11, 13, 0, 15),      # S2
        (11, 3, 5, 8, 2, 15, 10, 13, 14, 1, 7, 4, 12, 9, 6, 0),      # S3
        (12, 8, 2, 1, 13, 4, 15, 6, 7, 0, 10, 5, 3, 14, 9, 11),      # S4
        (7, 15, 5, 10, 8, 1, 6, 13, 0, 9, 3, 14, 11, 4, 2, 12),      # S5
        (5, 13, 15, 6, 9, 2, 12, 10, 11, 7, 8, 1, 4, 3, 14, 0),      # S6
        (8, 14, 2, 5, 6, 9, 1, 12, 15, 4, 11, 0, 13, 10, 3, 7),      # S7
        (1, 7, 14, 13, 0, 5, 8, 3, 4, 15, 10, 6, 9, 12, 11, 2)       # S8
    )

    # Функция раунда (выполняет преобразование 32-битного блока)
    def round_func(in_val, round_key):
        # 1. Сложение с ключом по модулю 2^32
        temp = (in_val + round_key) % (2**32)

        # 2. Замена по таблице S-box
        # Разбиваем 32 бита на 8 тетрад (по 4 бита)
        output = 0
        for i in range(8):
            # Извлекаем 4 бита. Сдвиг на 4*i
            nibble = (temp >> (4 * i)) & 0xF
            # Замена
            subbed = S_BOX[i][nibble]
            # Записываем обратно
            output |= (subbed << (4 * i))

        # 3. Циклический сдвиг влево на 11
        # ((x << 11) | (x >> 21)) & 0xFFFFFFFF обеспечивает 32-битную маску
        output = ((output << 11) | (output >> 21)) & 0xFFFFFFFF
        return output

    # --- 3. Основной цикл обработки блоков (сеть Фейстеля) ---

    result_bits = []

    # Разбиваем поток на блоки по 64 бита
    blocks = [bits_padded[i:i+64] for i in range(0, len(bits_padded), 64)]

    for block in blocks:
        # Преобразуем битовую строку в число
        block_int = int(block, 2)

        # Разделяем на левую (A) и правую (B) части по 32 бита
        # block = (A << 32) | B
```

```
        B = block_int & 0xFFFFFFFF          # Правая часть (N1)
        A = (block_int >> 32) & 0xFFFFFFFF  # Левая часть (N2)

        # Выполняем 32 раунда
        for k_idx in key_indices:
            k = subkeys[k_idx]

            # Основное уравнение раунда:
            # R_i+1 = L_i XOR RoundFunc(R_i, K)
            # L_i+1 = R_i

            func_res = round_func(B, k)
            new_B = A ^ func_res # XOR
            new_A = B            # Сдвиг (старое B становится новым A)

            A, B = new_A, new_B

        final_block_int = (B << 32) | A

        # Преобразуем обратно в 64 бита
        result_bits.append(f'{final_block_int:064b}')

    # --- 4. Формирование ответа ---

    full_output = "".join(result_bits)

    return full_output

def gost_28147_89_gamming(text: str, key: int, iv: int) -> str:
    """
    Реализует режим Гаммирования по ГОСТ 28147-89.

    Аргументы:
    text (str): Текст (А-Я) или поток битов.
    key (int): 256-битный ключ.
    iv (int): 64-битный инициализирующий вектор (синхропосылка).
    encrypt (bool): Флаг режима (для гаммирования логика симметрична).

    Возвращает:
    str: Поток битов.
    """

    # --- 1. Подготовка данных ---

    # Проверка: вход это биты или текст
    is_bit_stream = all(c in '01' for c in text) if text else False
    if is_bit_stream:
        bits = text
    else:
        bits = string_to_bits(remove_special_chars(text))
```

```
# --- 2. Константы и Настройки ГОСТ ---

# Константы для обновления IV (C1, C2) согласно ГОСТ 28147-89
# C1 = 16843012 (0x01010104), C2 = 16843009 (0x01010101)
C1 = 0x01010104
C2 = 0x01010101

# Таблица замен (S-box)
S_BOX = [1, 15, 13, 0, 5, 7, 10, 4, 9, 2, 3, 14, 6, 11, 8, 12]

# Разбиваем 256-битный ключ на 8 подключей по 32 бита
subkeys = [(key >> (32 * i)) & 0xFFFFFFFF for i in range(8)]

# В режиме гаммирования для выработки гаммы всегда используется
# порядок ключей как при простой замене в режиме ШИФРОВАНИЯ.
# (3 раза 0..7, 1 раз 7..0)
key_indices = list(range(8)) * 3 + list(range(7, -1, -1))

# Внутренняя функция шифрования одного 64-битного блока (режим простой замены)
# Она нужна для генерации Гаммы из IV
def encrypt_block_gost(block_int_64):
    # Разделяем IV на N1 (младшие, B) и N2 (старшие, A)
    n1 = block_int_64 & 0xFFFFFFFF          # Правая часть
    n2 = (block_int_64 >> 32) & 0xFFFFFFFF  # Левая часть

    for k_idx in key_indices:
        round_key = subkeys[k_idx]

        # (N1 + K) mod 2^32
        temp = (n1 + round_key) % (2**32)

        # Замена S-box
        output_s = 0
        for i in range(8):
            nibble = (temp >> (4 * i)) & 0xF
            subbed = S_BOX[nibble]
            output_s |= (subbed << (4 * i))

        # Циклический сдвиг влево на 11
        output_shifted = ((output_s << 11) | (output_s >> 21)) & 0xFFFFFFFF

        # Основной шаг сети Фейстеля
        # new_N1 = N2 ^ f(N1)
        # new_N2 = N1
        func_res = output_shifted
        new_n1 = n2 ^ func_res
        new_n2 = n1

        n2, n1 = new_n2, new_n1

    # Сборка результата (Гаммы)
```

```
        # Обратите внимание: в конце простой замены (N2, N1)
        return (n2 << 32) | n1

    # --- 3. Процесс Гаммирования ---

    result_bits = []

    # Работаем с копией IV, чтобы менять её в цикле
    current_iv_1 = iv & 0xFFFFFFFF          # N1 (младшие)
    current_iv_2 = (iv >> 32) & 0xFFFFFFFF  # N2 (старшие)
    current_iv_64 = encrypt_block_gost( (current_iv_2 << 32) | current_iv_1)

    current_iv_1 = current_iv_64 & 0xFFFFFFFF          # N1 (младшие)
    current_iv_2 = (current_iv_64 >> 32) & 0xFFFFFFFF  # N2 (старшие)

    # Обрабатываем входной поток частями по 64 бита (или меньше для последнего
куска)

    for i in range(0, len(bits), 64):
        chunk = bits[i:i+64]
        chunk_len = len(chunk)
        chunk_val = int(chunk, 2)

        # 1. Генерируем блок гаммы из текущего IV
        # Собираем текущий IV в 64-битное число для функции шифрования
        current_iv_1 = (current_iv_1 + C2) % (2**32)
        current_iv_2 = (current_iv_2 + C1) % (2**32-1)

        current_iv_64 = (current_iv_2 << 32) | current_iv_1

        # Получаем 64 бита гаммы (шифруем IV)
        gamma_block = encrypt_block_gost(current_iv_64)

        # 3. Накладываем Гамму на текст (XOR)
        # Если кусок текста меньше 64 бит, берем только старшие биты гаммы
        # Обрезаем гамму до длины текста

        # Чтобы сделать XOR корректно для обрезанного блока:
        if chunk_len < 64:
            # Форматируем гамму в строку и берем срез
            gamma_bits_str = f'{gamma_block:064b}'
            # Берем столько бит гаммы, сколько есть текста (слева направо по
потоку)
            gamma_part_int = int(gamma_bits_str[:chunk_len], 2)
            result_int = chunk_val ^ gamma_part_int

            # Добавляем в результат
            result_bits.append(format(result_int, f'0{chunk_len}b'))

        else:
            # Полный блок
```

```
            result_int = chunk_val ^ gamma_block
            result_bits.append(f'{result_int:064b}')



    return "".join(result_bits)

#####################################################
# Имплементация функций по шифрам Шеннона и Вернама

def generator_T_i(a, c, T_0, m, length):
    current = T_0
    counter = 0
    while counter <= length:
        # print(f"({a}*{current} + {c}) % {m} = ", end='')
        current = (a * current + c) % m
        # print(current)
        counter += 1
        yield current

def shenon(input_text: str, encrypt=True, a: int = 13, c: int = 7, T_0: int = 2):
    '''
    :param: input_text - Текст для шифрования - в символьной репрезентации (в т.ч.
со знаками препинаний)
    :param: a - константа
    :param: c - константа
    :param: T_0 - исходная величина для порождающего числа

    T(i+1) = (a * T(i) + c) mod m

    Период гаммы зависит от a, c и максимален при следующих условиях:
    - c - нечетное
    - a (mod 4) = 1

    Если гамма СИМВОЛЬНАЯ, то:
    - m - мощность алфавита
    - a - нечетное число
    - c - (c,m) = 1 (Взаимно простые)
    - b = a - 1 - кратно p для каждого простого p, делителя m
    - b кратно 4, если m кратно 4
    '''

    result = []

    if set(input_text) == set('01'):
        return
    m = len(ALPHABET)
    input_text = remove_special_chars(input_text)
    input_text = list(map(lambda x: ALPHABET.find(x), input_text))

    T_n = generator_T_i(a, c, T_0, m, length=len(input_text)-1)
```

```
    for id, gamma in enumerate(T_n):
        if encrypt:
            temp = (input_text[id] + gamma) % m
        else:
            temp = (input_text[id] - gamma) % m

        result.append( ALPHABET[temp] )

    return ''.join(result)

def binary_shenon(input_text: str, a: int = 13, c: int = 7, T_0: int = 2):
    '''
    :param: input_text - Текст для шифрования - в бинарной репрезентации или в
символьной
    :param: a - константа
    :param: c - константа
    :param: T_0 - исходная величина для порождающего числа

    T(i+1) = (a * T(i) + c) mod m

    Период гаммы зависит от a, c и максимален при следующих условиях:
    - c - нечетное
    - a (mod 4) = 1

    Если гамма СИМВОЛЬНАЯ, то:
    - m - мощность алфавита
    - a - нечетное число
    - c - (c,m) = 1 (Взаимно простые)
    - b = a - 1 - кратно p для каждого простого p, делителя m
    - b кратно 4, если m кратно 4
    '''

    result = []

    if not set(input_text) == set('01'):
        input_text = string_to_bits(remove_special_chars(input_text))
    S = 5
    m = 2**S
    # 5 - длина машинного слова в битах

    T_n = generator_T_i(a, c, T_0, m, length=len(input_text)-1)
    letter_bits = []

    for id, bit in enumerate(input_text):
        if id % S == 0:
            gamma = next(T_n)
            gamma = bin(gamma)[2:].zfill(S)
            if id > 0:
                binary = ''.join(letter_bits)
                result.append( ALPHABET[int( binary, 2)] )
```

```
                letter_bits = []

        new_bit = int(bit) ^ int(gamma[id % S])

        letter_bits.append(str(new_bit))

    if letter_bits:
        binary = ''.join(letter_bits)
        result.append( ALPHABET[int( binary, 2)] )

    return ''.join(result)


def vernam(input_text: str, key: str, encrypt=True):
    '''
    :param: input_text - Текст для шифрования - в символьной репрезентации (в т.ч.
со знаками препинаний)
    '''

    result = []

    if set(input_text) == set('01'):
        return
    m = len(ALPHABET)
    input_text = remove_special_chars(input_text)

    key = key.upper()
    length = len(key)

    for id, letter in enumerate(input_text):
        if encrypt:
            temp = (ALPHABET.find(letter) + ALPHABET.find(key[id % length])) % m
        else:
            temp = (ALPHABET.find(letter) - ALPHABET.find(key[id % length])) % m

        result.append( ALPHABET[temp] )

    return ''.join(result)

def binary_vernam(input_text: str, key: str):
    '''
    :param: input_text - Текст для шифрования - в бинарной репрезентации или в
символьной
    '''

    result = []

    if not set(input_text) == set('01'):
        input_text = string_to_bits(remove_special_chars(input_text))
    S = 5
    m = 2**S
```

```
    # 5 - длина машинного слова в битах
    key = string_to_bits(key.upper())
    length = len(key)

    letter_bits = []

    for id, bit in enumerate(input_text):
        new_bit = int(bit) ^ int(key[id % length])
        letter_bits.append(str(new_bit))

        if id > 0 and id % S == -1 % S:
            binary = ''.join(letter_bits)
            result.append( ALPHABET[int( binary, 2)] )
            letter_bits = []

    if letter_bits:
        binary = ''.join(letter_bits)
        result.append( ALPHABET[int( binary, 2)] )

    return ''.join(result)

#####################################################
# Тесты

def test_result(text, decrypted: str):
    if set(decrypted) == set('01'):
        decrypted = bits_to_string(decrypted)
    if set(text) == set('01'):
        text = bits_to_string(text)
    result = standartise(text) in standartise(decrypted)
    print(f"Тест {'пройден успешно '+OK_SYMBOL if result else 'провален!
'+NOT_OK_SYMBOL}")
    if not result:
        print(f"Оригинал: {standartise(text)}")
        print(f"Получили: {standartise(decrypted)}")

    return result

# Тесты для ГОСТ 28147-89

def test_GOST_gamming_example():
    key = 0xFFEEDDCCBBAA9988776655443322110 0F0F1F2F3F4F5F6F7F8F9FAFBFCFDFEFF
    iv = 0x1234567890ABCEF1
    text = 0x1234567890ABCEF123456789 #ABCDEF01
    text = bin(text)[2:]

    print(f"Исходный текст: {textwrap.shorten(standartise(text), width=WIDTH,
placeholder=PLACEHOLDER)}, Длина: {len(text)} битов")

    encrypted = gost_28147_89_gamming(text, key, iv)
```

```
    print("Зашифрованный  ", textwrap.shorten(standartise(encrypted), width=WIDTH,
placeholder=PLACEHOLDER))

    decrypted = gost_28147_89_gamming(encrypted, key, iv)
    print("Расшифрованный ", textwrap.shorten(standartise(decrypted), width=WIDTH,
placeholder=PLACEHOLDER))

    return test_result(text, decrypted)

def test_gamming_1():
    key = generateLargePrime(256)
    iv = generateLargePrime(64)
    # text = "ЕГОР Я НЕ БУДУ РИСОВАТЬ РЫБУ "
    text = PROVERB
    print(f"Исходный текст: {textwrap.shorten(text.strip(), width=WIDTH,
placeholder=PLACEHOLDER)}, Длина: {len(remove_special_chars(text))} символов")

    text = remove_special_chars(text)
    print(f"Биты из исходного текста:
{textwrap.shorten(standartise(string_to_bits(text)), width=WIDTH,
placeholder=PLACEHOLDER)}")

    # Проверка
    encrypted = gost_28147_89_gamming(text, key, iv)
    print("Зашифрованный            ", textwrap.shorten(standartise(encrypted),
width=WIDTH, placeholder=PLACEHOLDER))
    print("Зашифрованный (буквами)  ",
textwrap.shorten(standartise(bits_to_string(encrypted)), width=WIDTH,
placeholder=PLACEHOLDER))

    decrypted = gost_28147_89_gamming(encrypted, key, iv)
    print("Расшифрованный           ", textwrap.shorten(standartise(decrypted),
width=WIDTH, placeholder=PLACEHOLDER))
    print("Расшифрованный (буквами) ",
textwrap.shorten(standartise(bits_to_string(decrypted)), width=WIDTH,
placeholder=PLACEHOLDER))

    return test_result(text, decrypted)

def test_simple_sub_1():
    key = generateLargePrime(256)
    # text = "ЕГОР Я НЕ БУДУ РИСОВАТЬ РЫБУ "
    text = PROVERB
    print(f"Исходный текст: {textwrap.shorten(text, width=WIDTH,
placeholder=PLACEHOLDER)}, Длина: {len(remove_special_chars(text))} символов")

    encrypted = gost_28147_89_simple_sub(text, key)
    print("Зашифрованный            ", textwrap.shorten(standartise(encrypted),
width=WIDTH, placeholder=PLACEHOLDER))
```

```
    print("Зашифрованный (буквами)  ",
textwrap.shorten(standartise(bits_to_string(encrypted)), width=WIDTH,
placeholder=PLACEHOLDER))

    decrypted = gost_28147_89_simple_sub(encrypted, key, False)
    print("Расшифрованный           ", textwrap.shorten(standartise(decrypted),
width=WIDTH, placeholder=PLACEHOLDER))
    print("Расшифрованный (буквами) ",
textwrap.shorten(standartise(bits_to_string(decrypted)), width=WIDTH,
placeholder=PLACEHOLDER))

    return test_result(text, decrypted)

# Тесты для ГОСТ 28147-89 (текст 1000+ символов)

def test_gamming_2():
    key = generateLargePrime(256)
    iv  = generateLargePrime(64)
    text = LONG_TEXT
    print(f"Исходный текст: {textwrap.shorten(text.strip(), width=WIDTH,
placeholder=PLACEHOLDER)}, Длина: {len(remove_special_chars(text))} символов")

    text = remove_special_chars(text)
    print(f"Биты из исходного текста:
{textwrap.shorten(standartise(string_to_bits(text)), width=WIDTH,
placeholder=PLACEHOLDER)}")

    # Проверка
    encrypted = gost_28147_89_gamming(text, key, iv)
    print("Зашифрованный            ", textwrap.shorten(standartise(encrypted),
width=WIDTH, placeholder=PLACEHOLDER))
    print("Зашифрованный (буквами)  ",
textwrap.shorten(standartise(bits_to_string(encrypted)), width=WIDTH,
placeholder=PLACEHOLDER))

    decrypted = gost_28147_89_gamming(encrypted, key, iv)
    print("Расшифрованный           ", textwrap.shorten(standartise(decrypted),
width=WIDTH, placeholder=PLACEHOLDER))
    print("Расшифрованный (буквами) ",
textwrap.shorten(standartise(bits_to_string(decrypted)), width=WIDTH,
placeholder=PLACEHOLDER))

    return test_result(text, decrypted)

def test_simple_sub_2():
    key = generateLargePrime(256)
    text = LONG_TEXT

    print(f"Исходный текст: {textwrap.shorten(text, width=WIDTH,
placeholder=PLACEHOLDER)}, Длина: {len(remove_special_chars(text))} символов")
```

```
    encrypted = gost_28147_89_simple_sub(text, key)
    print("Зашифрованный            ", textwrap.shorten(standartise(encrypted),
width=WIDTH, placeholder=PLACEHOLDER))
    print("Зашифрованный (буквами)  ",
textwrap.shorten(standartise(bits_to_string(encrypted)), width=WIDTH,
placeholder=PLACEHOLDER))

    decrypted = gost_28147_89_simple_sub(encrypted, key, False)
    print("Расшифрованный           ", textwrap.shorten(standartise(decrypted),
width=WIDTH, placeholder=PLACEHOLDER))
    print("Расшифрованный (буквами) ",
textwrap.shorten(standartise(bits_to_string(decrypted)), width=WIDTH,
placeholder=PLACEHOLDER))

    return test_result(text, decrypted)

# Тесты для Шеннона и Вернама

def test_shenon_str():
    a, c, T_0 = 13, 7, 2
    # text = "ЕГОР Я НЕ БУДУ РИСОВАТЬ РЫБУ "
    text = PROVERB
    print(f"Исходный текст: {textwrap.shorten(text.strip(), width=WIDTH,
placeholder=PLACEHOLDER)}, Длина: {len(remove_special_chars(text))} символов")

    text = remove_special_chars(text)

    # Проверка
    encrypted = shenon(text, a=a, c=c, T_0=T_0)
    print("Зашифрованный            ", textwrap.shorten(standartise(encrypted),
width=WIDTH, placeholder=PLACEHOLDER))

    decrypted = shenon(encrypted, a=a, c=c, T_0=T_0, encrypt=False)
    print("Расшифрованный           ", textwrap.shorten(standartise(decrypted),
width=WIDTH, placeholder=PLACEHOLDER))

    return test_result(text, decrypted)

def test_shenon_bin():
    a, c, T_0 = 13, 7, 2
    # text = "ЕГОР Я НЕ БУДУ РИСОВАТЬ РЫБУ "
    text = PROVERB
    print(f"Исходный текст: {textwrap.shorten(text, width=WIDTH,
placeholder=PLACEHOLDER)}, Длина: {len(remove_special_chars(text))} символов")

    encrypted = binary_shenon(text, a=a, c=c, T_0=T_0)
    print("Зашифрованный            ", textwrap.shorten(standartise(encrypted),
width=WIDTH, placeholder=PLACEHOLDER))

    decrypted = binary_shenon(encrypted, a=a, c=c, T_0=T_0)
```

```
    print("Расшифрованный           ", textwrap.shorten(standartise(decrypted),
width=WIDTH, placeholder=PLACEHOLDER))

    return test_result(text, decrypted)


def test_vernam_str():
    key = "КРИПТОГРАФИЯ"
    # text = "ЕГОР Я НЕ БУДУ РИСОВАТЬ РЫБУ "
    text = PROVERB
    print(f"Исходный текст: {textwrap.shorten(text.strip(), width=WIDTH,
placeholder=PLACEHOLDER)}, Длина: {len(remove_special_chars(text))} символов")

    text = remove_special_chars(text)

    # Проверка
    encrypted = vernam(text, key)
    print("Зашифрованный            ", textwrap.shorten(standartise(encrypted),
width=WIDTH, placeholder=PLACEHOLDER))

    decrypted = vernam(encrypted, key, encrypt=False)
    print("Расшифрованный           ", textwrap.shorten(standartise(decrypted),
width=WIDTH, placeholder=PLACEHOLDER))

    return test_result(text, decrypted)

def test_vernam_bin():
    key = "КРИПТОГРАФИЯ"
    # text = "ЕГОР Я НЕ БУДУ РИСОВАТЬ РЫБУ "
    text = PROVERB
    print(f"Исходный текст: {textwrap.shorten(text, width=WIDTH,
placeholder=PLACEHOLDER)}, Длина: {len(remove_special_chars(text))} символов")

    encrypted = binary_vernam(text, key)
    print("Зашифрованный            ", textwrap.shorten(standartise(encrypted),
width=WIDTH, placeholder=PLACEHOLDER))

    decrypted = binary_vernam(encrypted, key)
    print("Расшифрованный           ", textwrap.shorten(standartise(decrypted),
width=WIDTH, placeholder=PLACEHOLDER))

    return test_result(text, decrypted)

# Тесты для Шеннона и Вернама (текст 1000+ символов)

def test_long_text_shenon_str():
    a, c, T_0 = 13, 7, 2
    # text = "ЕГОР Я НЕ БУДУ РИСОВАТЬ РЫБУ "
    text = LONG_TEXT
    print(f"Исходный текст: {textwrap.shorten(text.strip(), width=WIDTH,
placeholder=PLACEHOLDER)}, Длина: {len(remove_special_chars(text))} символов")
```

```
    text = remove_special_chars(text)

    # Проверка
    encrypted = shenon(text, a=a, c=c, T_0=T_0)
    print("Зашифрованный            ", textwrap.shorten(standartise(encrypted),
width=WIDTH, placeholder=PLACEHOLDER))

    decrypted = shenon(encrypted, a=a, c=c, T_0=T_0, encrypt=False)
    print("Расшифрованный           ", textwrap.shorten(standartise(decrypted),
width=WIDTH, placeholder=PLACEHOLDER))

    return test_result(text, decrypted)

def test_long_text_shenon_bin():
    a, c, T_0 = 13, 7, 2
    # text = "ЕГОР Я НЕ БУДУ РИСОВАТЬ РЫБУ "
    text = LONG_TEXT
    print(f"Исходный текст: {textwrap.shorten(text, width=WIDTH,
placeholder=PLACEHOLDER)}, Длина: {len(remove_special_chars(text))} символов")

    encrypted = binary_shenon(text, a=a, c=c, T_0=T_0)
    print("Зашифрованный            ", textwrap.shorten(standartise(encrypted),
width=WIDTH, placeholder=PLACEHOLDER))

    decrypted = binary_shenon(encrypted, a=a, c=c, T_0=T_0)
    print("Расшифрованный           ", textwrap.shorten(standartise(decrypted),
width=WIDTH, placeholder=PLACEHOLDER))

    return test_result(text, decrypted)


def test_long_text_vernam_str():
    key = "КРИПТОГРАФИЯ"
    # text = "ЕГОР Я НЕ БУДУ РИСОВАТЬ РЫБУ "
    text = LONG_TEXT
    print(f"Исходный текст: {textwrap.shorten(text.strip(), width=WIDTH,
placeholder=PLACEHOLDER)}, Длина: {len(remove_special_chars(text))} символов")

    text = remove_special_chars(text)

    # Проверка
    encrypted = vernam(text, key)
    print("Зашифрованный            ", textwrap.shorten(standartise(encrypted),
width=WIDTH, placeholder=PLACEHOLDER))

    decrypted = vernam(encrypted, key, encrypt=False)
    print("Расшифрованный           ", textwrap.shorten(standartise(decrypted),
width=WIDTH, placeholder=PLACEHOLDER))

    return test_result(text, decrypted)
```

```
def test_long_text_vernam_bin():
    key = "КРИПТОГРАФИЯ"
    # text = "ЕГОР Я НЕ БУДУ РИСОВАТЬ РЫБУ "
    text = LONG_TEXT
    print(f"Исходный текст: {textwrap.shorten(text, width=WIDTH,
placeholder=PLACEHOLDER)}, Длина: {len(remove_special_chars(text))} символов")

    encrypted = binary_vernam(text, key)
    print("Зашифрованный            ", textwrap.shorten(standartise(encrypted),
width=WIDTH, placeholder=PLACEHOLDER))

    decrypted = binary_vernam(encrypted, key)
    print("Расшифрованный           ", textwrap.shorten(standartise(decrypted),
width=WIDTH, placeholder=PLACEHOLDER))

    return test_result(text, decrypted)

#####################################################
# API

def option_encrypt(encrypt_func: Callable, message_type: str = "str", key_type: str
= "int"):
    """
    Asks User for a string to be encrypted and a key

    Calls given Encrypting function

    Adds the result to the Clipboard
    """
    if message_type == "str":
        my_Message = input("Введите сообщение для шифрования: (символы)\n")
    else:
        my_Message = input("Введите сообщение для шифрования: (поток битов)\n")
    my_Message = my_Message[:len(my_Message)] # Removing '\n' symbol

    if encrypt_func.__name__ == 'shenon' or encrypt_func.__name__ ==
'binary_shenon':
        a_param   = int(input("Введите параметр a: (int)\n"))
        c_param   = int(input("Введите параметр c: (int)\n"))
        T_0_param = int(input("Введите параметр T_0: (int)\n"))
        Encrypted_message: str | None = encrypt_func(input_text=my_Message,
a=a_param, c=c_param, T_0=T_0_param)

    elif encrypt_func.__name__ == 'gost_28147_89_gamming':
        key_param   = input("Введите параметр key (256 бит): (int)\n")
        try:
            key_param = int(key_param)
        except Exception:
            key_param = int(key_param, 16)
        iv_param    = input("Введите параметр IV   (64): (int)\n")
```

```
        try:
            iv_param = int(iv_param)
        except Exception:
            iv_param = int(iv_param, 16)
        Encrypted_message: str | None =
bits_to_string(encrypt_func(text=my_Message, key=key_param, iv=iv_param))

    else:
        if key_type == "int":
            my_key = int(input("Введите ключ: (int)\n"))
        else:
            my_key = input("Введите ключ: (str)\n")

        Encrypted_message: str | None = encrypt_func(input_text=my_Message,
key=my_key)

    if Encrypted_message != None:
        print(f"Зашифрованное сообщение:\n{Encrypted_message}")

def option_decrypt(decrypt_func: Callable, message_type: str = "str", key_type: str
= "int"):
    """
    Asks User for a string to be decrypted and a key

    Calls given Decrypting function

    Adds the result to the Clipboard
    """
    if message_type == "str":
        my_Message = input("Введите сообщение для расшифрования: (символы)\n")
    else:
        my_Message = input("Введите сообщение для расшифрования: (поток битов)\n")
    my_Message = my_Message[:len(my_Message)] # Removing '\n' symbol

    if decrypt_func.__name__ == 'shenon' or decrypt_func.__name__ ==
'binary_shenon':
        a_param   = int(input("Введите параметр a: (int)\n"))
        c_param   = int(input("Введите параметр c: (int)\n"))
        T_0_param = int(input("Введите параметр T_0: (int)\n"))
        Decrypted_message: str | None = decrypt_func(input_text=my_Message,
a=a_param, c=c_param, T_0=T_0_param, encrypt=False)

    elif decrypt_func.__name__ == 'vernam':
        my_key = input("Введите ключ: (str)\n")
        Decrypted_message: str | None = decrypt_func(input_text=my_Message,
key=my_key, encrypt=False)

    elif decrypt_func.__name__ == 'gost_28147_89_gamming':
        key_param   = input("Введите параметр key (256 бит): (int)\n")
        try:
            key_param = int(key_param)
```

```
        except Exception:
            key_param = int(key_param, 16)
        iv_param    = input("Введите параметр IV   (64): (int)\n")
        try:
            iv_param = int(iv_param)
        except Exception:
            iv_param = int(iv_param, 16)
        Decrypted_message: str | None =
bits_to_string(decrypt_func(text=my_Message, key=key_param, iv=iv_param))

    else:
        if key_type == "int":
            my_key = int(input("Введите ключ: (int)\n"))
        else:
            my_key = input("Введите ключ: (str)\n")

        Decrypted_message: str | None = decrypt_func(input_text=my_Message,
key=my_key)
    if Decrypted_message != None:
        print(f"Расшифрованное сообщение:\n{Decrypted_message}")

def do_tests():
    """
    Проводим тесты
    """
    succeeded = 0
    total = 0
    for id, f in enumerate([test_GOST_gamming_example,
                            test_gamming_1, test_simple_sub_1,
                            test_gamming_2, test_simple_sub_2,
                            test_shenon_str, test_shenon_bin,
                            test_vernam_str, test_vernam_bin,
                            test_long_text_shenon_str, test_long_text_shenon_bin,
                            test_long_text_vernam_str, test_long_text_vernam_bin]):
        print(f"{'#'*75}\nПроводим тест №{id+1} ({f.__name__})...\n")

        if f():
            succeeded += 1
        else:
            print(f.__name__)
            os.system("pause")
        total += 1

        print(f"\nТест №{id+1} завершен!\n")

    print(f"{'#'*75}\nИтоговый успех тестирования: {succeeded}/{total} =
{100*succeeded/total:.1f} %")

#####################################################
# Graphics
```

```
def clear_screen_ansi():
    """Очищает весь экран и перемещает курсор в начало"""
    print("\033[2J\033[H", end="", flush=True)

def clear_terminal():
    """Кроссплатформенная очистка терминала"""
    system = platform.system()

    if system == "Windows":
        os.system("cls")
    else:  # Linux, macOS, Unix
        os.system("clear")

def menu_shenon():
    option: int = -1
    while (True):
        try:
            clear_screen_ansi()
        except Exception:
            try:
                clear_terminal()
            except Exception:
                return
        print("""Шифрование шифром Шеннона
    Выберите:
    0) Выйти назад...
    1) Зашифровать сообщение  (символьное)
    2) Расшифровать сообщение (символьное)
    3) Зашифровать сообщение  (бинарное)
    4) Расшифровать сообщение (бинарное)""")

        try:
            option = int(input()[:1])
        except ValueError:
            continue
        try:
            clear_screen_ansi()
        except Exception:
            try:
                clear_terminal()
            except Exception:
                return
        if option == 0: break
        else:
            if option == 1:   option_encrypt(encrypt_func=shenon)
            elif option == 2: option_decrypt(decrypt_func=shenon)
            elif option == 3: option_encrypt(decrypt_func=binary_shenon,
message_type = "bin")
            elif option == 4: option_decrypt(decrypt_func=binary_shenon,
message_type = "bin")
            os.system("pause")
```

```
def menu_vernam():
    option: int = -1
    while (True):
        try:
            clear_screen_ansi()
        except Exception:
            try:
                clear_terminal()
            except Exception:
                return
        print("""Шифрование шифром Вернама
    Выберите:
    0) Выйти назад...
    1) Зашифровать сообщение  (символьное)
    2) Расшифровать сообщение (символьное)
    3) Зашифровать сообщение  (бинарное)
    4) Расшифровать сообщение (бинарное)""")

        try:
            option = int(input()[:1])
        except ValueError:
            continue
        try:
            clear_screen_ansi()
        except Exception:
            try:
                clear_terminal()
            except Exception:
                return
        if option == 0: break
        else:
            if option == 1:   option_encrypt(encrypt_func=vernam, key_type="str")
            elif option == 2: option_decrypt(decrypt_func=vernam, key_type="str")
            elif option == 3: option_encrypt(decrypt_func=binary_vernam,
message_type = "bin", key_type="str")
            elif option == 4: option_decrypt(decrypt_func=binary_vernam,
message_type = "bin", key_type="str")
            os.system("pause")

def menu_gost():
    option: int = -1
    while (True):
        try:
            clear_screen_ansi()
        except Exception:
            try:
                clear_terminal()
            except Exception:
                return
        print("""Шифрование шифром ГОСТ 28147-89
```

```
    Выберите:
    0) Выйти назад...
    1) Зашифровать сообщение  (символьное)
    2) Расшифровать сообщение (символьное)
    3) Зашифровать сообщение  (бинарное)
    4) Расшифровать сообщение (бинарное)""")

        try:
            option = int(input()[:1])
        except ValueError:
            continue
        try:
            clear_screen_ansi()
        except Exception:
            try:
                clear_terminal()
            except Exception:
                return
        if option == 0: break
        else:
            if option == 1:   option_encrypt(encrypt_func=gost_28147_89_gamming)
            elif option == 2: option_decrypt(decrypt_func=gost_28147_89_gamming)
            elif option == 3: option_encrypt(encrypt_func=gost_28147_89_gamming,
message_type="bin")
            elif option == 4: option_decrypt(decrypt_func=gost_28147_89_gamming,
message_type="bin")
            os.system("pause")

def menu_tests():
    do_tests()
    os.system("pause")

#####################################################
# MAIN

def main():
    option: int = -1
    while (True):
        try:
            clear_screen_ansi()
        except Exception:
            try:
                clear_terminal()
            except Exception:
                return
        print("""Шифрование шифрами гаммирования
    Выберите:
    0) Закрыть программу...
    1) Использовать Шифр Шеннона
    2) Использовать Шифр Вернама
    3) Использовать Шифр Гаммирования по ГОСТ 28147-89
```

```
    4) Запустить тесты""")

        try:
            option = int(input()[:1])
        except ValueError:
            continue
        try:
            clear_screen_ansi()
        except Exception:
            try:
                clear_terminal()
            except Exception:
                return
        if option == 0: break
        else:
            if option == 1:   menu_shenon()
            elif option == 2: menu_vernam()
            elif option == 3: menu_gost()
            elif option == 4: menu_tests()

if __name__ == "__main__":

    main()
```